Chapter 9 - Training and Adapting Vision-Language Models

Learning Outcomes

By the end of this chapter, you should be able to:

- **Explain why vision-language models (VLMs) have not yet reached the level of text post-training** in RAG, especially how they summarise different files having, understand and, crucially, use the generated text.
- **Distinguish between methods that do not update model weights** (such as zero-shot, few-shot, and most RAG setups) and methods that do update model weights (such as supervised fine-tuning, LoRA, QLoRA, and preference optimization).
- **Explain when LoRA, QLoRA, DPO, and ORPO are a good choice**, and identify the main trade-offs in memory, performance, data requirements, and engineering complexity.
- **Describe how few-shot learning and RAG improve behavior without changing parameters**, and explain the intuition behind in-context learning and retrieval-based prompting.
- **Define the supervised fine-tuning objective** and explain how SFT changes model behavior by increasing the likelihood of desired outputs.
- **Explain LoRA and QLoRA**, including why they are popular, why low-rank updates are effective, and how quantization helps reduce memory requirements.
- **Construct and interpret preference-based training data**, using prompt, chosen, and rejected responses for methods such as DPO.
- **Explain how training can improve factuality, instruction following, style, and safety**, and distinguish between what different post-training methods can and cannot fix.
- **Describe the key online-consumer-learning concepts used in language-model alignment**, including policy, reward, return, advantage, baseline, KL regularization, and why they matter in practice.
- **Evaluate the trade-offs among data requirements, compute cost, hardware constraints, and expected gains** when choosing an adaptation method for a real VLM application.

License

Text, figures, and explanations:
© 2026 Ivan Gaibulloev. Licensed under **CC BY 4.0**.

Code cells:
© 2026 Ivan Gaibulloev. Licensed under the **MIT License**.

You are free to share, modify, and redistribute with attribution.

Overview

The chapter is organized according to the methods below. The opening sections build the shared, chosen-a-working-subset, and define a common evaluation format so that the methods can be compared fairly. Then each method is introduced in turn.

Method table and budget ladder

The table below is the reference you'll keep coming back to.

	Method	Method	Changes model weights?	Main cost	Typical use
1	Zero-shot	No	No	Tokens	Quick baseline, simple tasks
2	Few-shot	No	No	Tokens	Format, style, and edge-case steering
3	RAG	No	No	Tokens + infra	Open-book knowledge
4	SFT	Yes	Yes	GPU hours	Strong behavior change
5	LoRA / QLoRA	Yes	Yes	GPU hours	Efficient fine-tuning
6	DPO	Yes	Yes	GPU hours	Preference alignment
7	ORPO	Yes	Yes	GPU hours	Preference alignment

A concept is worth learning if it changes what the next project should:

- **Method 1** changes the way you write prompts.
- **Method 2** changes the number of examples in those prompts.
- **Methods 3 through 7** change the quality, freshness, or operational cost of your system.

Environment setup

The next cells install and import the libraries used in the lecture rest.

The most libraries are:

- `TOKENIZERS_PARALLELISM` to avoid the HF tokenizers warning
- `HF_HUB_DISABLE_TELEMETRY` to disable telemetry
- `SEED` for RT and GPU kernels
- `DEVICE` to select the device
- `DTYPE` for numerical precision during inference

The base model used here is the quant for Qwen3-VL-4b, inspires a sufficiently recent Transformers version. In practice, if an environment is stale, the safest fix is usually to update `transformers`, `accelerate`, `peft`, and `bitsandbytes` together.

```
%%capture
!pip install -U \
    "transformers>=4.57.0" \
    accelerate \
    peft \
    bitsandbytes \
    datasets \
    trl \
    sentencepiece \
    protobuf \
    "huggingface_hub>=0.34.0" \
    "safetensors>=0.4.3" \
    pillow \
    matplotlib \
    pandas \
    scikit-learn \
    qwen-vl-utils
```

```
import os
os.environ["TOKENIZERS_PARALLELISM"] = "false"
os.environ["HF_HUB_DISABLE_TELEMETRY"] = "1"

import random
import numpy as np
import torch

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)

print("torch:", torch.__version__)
print("cuda available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("device:", torch.cuda.get_device_name(0))
```

```
torch: 2.10.0+cu128
cuda available: True
device: Tesla T4
```

```
import transformers, peft, bitsandbytes, trl, datasets
print("transformers:", transformers.__version__)
print("peft:", peft.__version__)
print("bitsandbytes:", bitsandbytes.__version__)
print("trl:", trl.__version__)
print("datasets:", datasets.__version__)
```

```
transformers: 4.57.6
peft: 0.18.1
bitsandbytes: 0.49.2
trl: 0.24.0
datasets: 4.3.0
```

```
import torch

if torch.cuda.is_available():
    major, minor = torch.cuda.get_device_capability(0)
    BF16_OK = major >= 8
else:
    BF16_OK = False

DTYPE = torch.bfloat16 if BF16_OK else torch.float16
print("bf16 supported:", BF16_OK)
print("using dtype:", DTYPE)
```

```
bf16 supported: False
using dtype: torch.float16
```

```
import os
import gc
import time
import json
import random
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F

from datasets import load_dataset, Dataset
from PIL import Image, ImageDraw, ImageFont

from transformers import (
    AutoProcessor,
    BitsAndBytesConfig,
    Qwen3VLForConditionalGeneration,
)

from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
```

Configuration

This cell defines the notebook configuration.

A few flags are shortcuts are to `TRUE` by default because some sections are heavier:

- `RUN_LORA_TRAINING`
- `RUN_QLORA_TRAINING_DEMO`
- `RUN_TINY_TRAINING`
- `RUN_DPO_DEMO`

A typical workflow is:

- run `RUN_BASELINE_ONLY TRUE`
- inspect the baseline outputs
- then enable `RUN_TINY_TRAINING`
- finally enable `RUN_LORA_TRAINING` if desired

```
# ============================================================
# Configuration
# ============================================================

MODEL_ID = "Qwen/Qwen3-VL-4B-Instruct"

# Keep the first run small and cheap.
RUN_BASELINE_ONLY = True
RUN_TINY_TRAINING = False
RUN_LORA_TRAINING = False
RUN_QLORA_TRAINING_DEMO = False
RUN_DPO_DEMO = False

# Dataset sizes
TRAIN_SIZE = 256
EVAL_SIZE = 64

# Image resizing to control memory use
MAX_IMAGE_SIDE = 512

# Generation
MAX_NEW_TOKENS = 128

# Training
MAX_SEQ_LENGTH = 2048
PER_DEVICE_BATCH_SIZE = 1
GRAD_ACCUM_STEPS = 8
LEARNING_RATE = 2e-4
NUM_EPOCHS = 1

# LoRA
LORA_R = 16
LORA_ALPHA = 32
LORA_DROPOUT = 0.05

# Output directories
OUTPUT_DIR = Path("outputs")
ADAPTER_DIR = OUTPUT_DIR / "lora_adapter"
QLORA_ADAPTER_DIR = OUTPUT_DIR / "qlora_adapter"
DPO_ADAPTER_DIR = OUTPUT_DIR / "dpo_adapter"

OUTPUT_DIR.mkdir(exist_ok=True)

print("MODEL_ID:", MODEL_ID)
print("RUN_BASELINE_ONLY:", RUN_BASELINE_ONLY)
print("RUN_TINY_TRAINING:", RUN_TINY_TRAINING)
print("RUN_LORA_TRAINING:", RUN_LORA_TRAINING)
print("RUN_QLORA_TRAINING_DEMO:", RUN_QLORA_TRAINING_DEMO)
print("RUN_DPO_DEMO:", RUN_DPO_DEMO)
```

```
MODEL_ID: Qwen/Qwen3-VL-4B-Instruct
RUN_BASELINE_ONLY: True
RUN_TINY_TRAINING: False
RUN_LORA_TRAINING: False
RUN_QLORA_TRAINING_DEMO: False
RUN_DPO_DEMO: False
```

Choose a biologically meaningful working subset

The original dataset is much larger than what we need for repeated experiments in one notebook. We therefore construct a subset that is:

- balanced across species
- small enough to support repeated inference and tuning runs
- diverse enough to include different habitats, locations, and visually challenging cases

Here we choose **10 species** and sample a fixed number of images per species. Using a predefined shortlist rather than a random grab helps make the notebook more reproducible and more pedagogically coherent.

```
# ============================================================
# Choose a biologically meaningful working subset
# ============================================================

def choose_balanced_subset(df, species_col, n_species, n_per_species, seed):
    """
    Choose the most common species, then sample a fixed number from each.
    This keeps the subset balanced and reproducible.
    """
    counts = df[species_col].value_counts()
    chosen = counts.head(n_species).index.tolist()

    parts = []
    for sp in chosen:
        sub = df[df[species_col] == sp]
        if len(sub) > n_per_species:
            sub = sub.sample(n_per_species, random_state=seed)
        parts.append(sub)

    out = pd.concat(parts).sample(frac=1.0, random_state=seed).reset_index(drop=True)
    return out, chosen

# Start from the full training split
full_df = ds["train"].to_pandas()

subset_df, chosen_species = choose_balanced_subset(
    full_df,
    species_col="species",
    n_species=10,
    n_per_species=40,
    seed=SEED,
)

print("Chosen species:")
for sp in chosen_species:
    print("-", sp)

print("\nSubset shape:", subset_df.shape)
print("\nSubset species counts:")
print(subset_df["species"].value_counts())
```

```
Chosen species:
- Ursus arctos horribilis
- Panthera leo
- Lynx rufus
- Lynx canadensis
- Canis lupus
- Vulpes vulpes
- Alces alces
- Rangifer tarandus
- Ovis dalli
- Ovis canadensis

Subset shape: (400, 7)

Subset species counts:
species
Ursus arctos horribilis    40
Panthera leo               40
Lynx rufus                 40
Lynx canadensis            40
Canis lupus                40
Vulpes vulpes              40
Alces alces                40
Rangifer tarandus          40
Ovis dalli                 40
Ovis canadensis            40
Name: count, dtype: int64
```

```
# Create a smaller working dataset for the rest of the notebook
ds = ds.filter(lambda x: x["species"] in chosen_species)

print(ds)
```

```
Filter: 100%|██████████| 1000/1000 [00:00<00:00, 1000.00 examples/s]
DatasetDict({
    train: Dataset({
        features: ['image_ref', 'image', 'split', 'uuid', 'location', 'species', 'genus', 'family', 'scientific_name', 'common_names', 'source'],
        num_rows: 400
    })
})
```

```
# Sanity check: inspect a few examples
for i in range(3):
    ex = ds["train"][i]
    print(f"\nExample {i}")
    print("species:", ex["species"])
    print("location:", ex["location"])
    print("image_ref:", ex["image_ref"])
```

```
Example 0
species: Ursus arctos horribilis
location: Alaska, USA
image_ref: {'path': 'data/train-00000-of-00001-3a8830e2f4406114.parquet', 'bytes': None}

Example 1
species: Ursus arctos horribilis
location: Alaska, USA
image_ref: {'path': 'data/train-00000-of-00001-3a8830e2f4406114.parquet', 'bytes': None}

Example 2
species: Ursus arctos horribilis
location: Alaska, USA
image_ref: {'path': 'data/train-00000-of-00001-3a8830e2f4406114.parquet', 'bytes': None}
```

Load the real Rare Species dataset

We load the real dataset from Hugging Face Datasets.

The goal is to end up with the same dataset interface as the toy dataset, so we can:

- inspect a few examples,
- reuse the same formatting helpers,
- and swap datasets without rewriting the rest of the notebook.

```
from datasets import load_dataset

ds = load_dataset("ivangabriele/rare-species", trust_remote_code=True)

print(ds)
```

```
README.md: 100%|██████████| 1.00k/1.00k [00:00<00:00, 3.31MB/s]
rare-species.py: 100%|██████████| 2.31k/2.31k [00:00<00:00, 8.31MB/s]
Downloading data: 100%|██████████| 1/1 [00:00<00:00,  1.00it/s]
Generating train split: 100%|██████████| 1000/1000 [00:00<00:00, 1000.00 examples/s]
DatasetDict({
    train: Dataset({
        features: ['image_ref', 'image', 'split', 'uuid', 'location', 'species', 'genus', 'family', 'scientific_name', 'common_names', 'source'],
        num_rows: 1000
    })
})
```

```
# Inspect a few examples
for i in range(3):
    ex = ds["train"][i]
    print(f"\nExample {i}")
    print("species:", ex["species"])
    print("location:", ex["location"])
    print("image_ref:", ex["image_ref"])
```

```
Example 0
species: Ursus arctos horribilis
location: Alaska, USA
image_ref: {'path': 'data/train-00000-of-00001-3a8830e2f4406114.parquet', 'bytes': None}

Example 1
species: Ursus arctos horribilis
location: Alaska, USA
image_ref: {'path': 'data/train-00000-of-00001-3a8830e2f4406114.parquet', 'bytes': None}

Example 2
species: Ursus arctos horribilis
location: Alaska, USA
image_ref: {'path': 'data/train-00000-of-00001-3a8830e2f4406114.parquet', 'bytes': None}
```

	scientific_name	common_names	species	genus	family	phylum	class	order	location	source
0	Ursus arctos horribilis	0302727	4 more	Ursus	Ursidae	Chordata	Mammalia	Carnivora	Alaska, USA	iNaturalist Open Data
1	Ursus arctos horribilis	0302727	4 more	Ursus	Ursidae	Chordata	Mammalia	Carnivora	Alaska, USA	iNaturalist Open Data
2	Ursus arctos horribilis	2686894	19283	Ursus	Ursidae	Chordata	Mammalia	Carnivora	Denmarkstræde, Rangittota, Aotearoa	iNaturalist Open Data
3	Ursus arctos horribilis	0302727	4 more	Ursus	Ursidae	Chordata	Mammalia	Carnivora	Kodiak Island, Alaska, USA	iNaturalist Open Data
4	Ursus arctos horribilis	0302727	4 more	Ursus	Ursidae	Chordata	Mammalia	Carnivora	Alaska, USA	iNaturalist Open Data

Choose a biologically meaningful working subset

The original dataset is much larger than what we need for repeated experiments in one notebook. We therefore construct a subset that is:

- balanced across species
- small enough to support repeated inference and tuning runs
- diverse enough to include different habitats, locations, and visually challenging cases

Here we choose **10 species** and sample a fixed number of images per species. Using a predefined shortlist rather than a random grab helps make the notebook more reproducible and more pedagogically coherent.

```
# ============================================================
# Choose a biologically meaningful working subset
# ============================================================

def choose_balanced_subset(df, species_col, n_species, n_per_species, seed):
    """
    Choose the most common species, then sample a fixed number from each.
    This keeps the subset balanced and reproducible.
    """
    counts = df[species_col].value_counts()
    chosen = counts.head(n_species).index.tolist()

    parts = []
    for sp in chosen:
        sub = df[df[species_col] == sp]
        if len(sub) > n_per_species:
            sub = sub.sample(n_per_species, random_state=seed)
        parts.append(sub)

    out = pd.concat(parts).sample(frac=1.0, random_state=seed).reset_index(drop=True)
    return out, chosen

# Start from the full training split
full_df = ds["train"].to_pandas()

subset_df, chosen_species = choose_balanced_subset(
    full_df,
    species_col="species",
    n_species=10,
    n_per_species=40,
    seed=SEED,
)

print("Chosen species:")
for sp in chosen_species:
    print("-", sp)

print("\nSubset shape:", subset_df.shape)
print("\nSubset species counts:")
print(subset_df["species"].value_counts())
```

```
Chosen species:
- Ursus arctos horribilis
- Panthera leo
- Lynx rufus
- Lynx canadensis
- Canis lupus
- Vulpes vulpes
- Alces alces
- Rangifer tarandus
- Ovis dalli
- Ovis canadensis

Subset shape: (400, 7)

Subset species counts:
species
Ursus arctos horribilis    40
Panthera leo               40
Lynx rufus                 40
Lynx canadensis            40
Canis lupus                40
Vulpes vulpes              40
Alces alces                40
Rangifer tarandus          40
Ovis dalli                 40
Ovis canadensis            40
Name: count, dtype: int64
```

```
# Create a smaller working dataset for the rest of the notebook
ds = ds.filter(lambda x: x["species"] in chosen_species)

print(ds)
```

```
Filter: 100%|██████████| 1000/1000 [00:00<00:00, 1000.00 examples/s]
DatasetDict({
    train: Dataset({
        features: ['image_ref', 'image', 'split', 'uuid', 'location', 'species', 'genus', 'family', 'scientific_name', 'common_names', 'source'],
        num_rows: 400
    })
})
```

```
# Sanity check: inspect a few examples
for i in range(3):
    ex = ds["train"][i]
    print(f"\nExample {i}")
    print("species:", ex["species"])
    print("location:", ex["location"])
    print("image_ref:", ex["image_ref"])
```

```
Example 0
species: Ursus arctos horribilis
location: Alaska, USA
image_ref: {'path': 'data/train-00000-of-00001-3a8830e2f4406114.parquet', 'bytes': None}

Example 1
species: Ursus arctos horribilis
location: Alaska, USA
image_ref: {'path': 'data/train-00000-of-00001-3a8830e2f4406114.parquet', 'bytes': None}

Example 2
species: Ursus arctos horribilis
location: Alaska, USA
image_ref: {'path': 'data/train-00000-of-00001-3a8830e2f4406114.parquet', 'bytes': None}
```

Load the Qwen VLM

We use `Qwen3VLForConditionalGeneration` throughout.

Why load in 4-bit?

A 4-bit load makes it possible to keep a strong base model in memory while leaving room for adapters, optimizer state, and batches on a single free-tier GPU. This is also the setup we will reuse later for QLoRA.

```
def load_base_processor_and_model(use_4bit=False, force_eager_sdpa=False):
    model_kwargs = {"device_map": "auto"}

    if use_4bit and torch.cuda.is_available():
        bnb_config = BitsAndBytesConfig(
            load_in_4bit=True,
            bnb_4bit_quant_type="nf4",
            bnb_4bit_compute_dtype=DTYPE,
            bnb_4bit_use_double_quant=True,
        )
        model_kwargs["quantization_config"] = bnb_config
    else:
        model_kwargs["dtype"] = DTYPE

    if force_eager_sdpa:
        model_kwargs["attn_implementation"] = "eager"

    processor = AutoProcessor.from_pretrained(MODEL_ID)
    model = Qwen3VLForConditionalGeneration.from_pretrained(MODEL_ID, **model_kwargs)
    model.eval()
    return processor, model

processor, model = load_base_processor_and_model(use_4bit=True)
print("Loaded model:", MODEL_ID)
```

```
`torch_dtype` is deprecated! Use `dtype` instead!
Loaded model: Qwen/Qwen3-VL-4B-Instruct
```

```
# Inspect the model's multimodal configuration
print("image_token_id:", model.config.image_token_id)
print("vision_start_token_id:", model.config.vision_start_token_id)
print("vision_end_token_id:", model.config.vision_end_token_id)
```

```
image_token_id: 151655
vision_start_token_id: 151652
vision_end_token_id: 151653
```

Shared utility functions

The next cell defines reusable utilities for:

- creating a consistent train and eval split,
- converting examples into chat format,
- and running generation on one sample.

Keeping the task definition and output format fixed is important because later comparisons should reflect the method itself rather than a hidden change in prompt wording or preprocessing.

```
import random

def make_train_eval_split(dataset, eval_size=64, seed=42):
    """Create a deterministic train/eval split."""
    n = len(dataset)
    eval_size = min(eval_size, n // 5)  # cap at 20% for very small datasets
    indices = list(range(n))
    rng = random.Random(seed)
    rng.shuffle(indices)
    eval_idx = indices[:eval_size]
    train_idx = indices[eval_size:]
    return dataset.select(train_idx), dataset.select(eval_idx)

def get_image(example, image_field="image"):
    """Return a PIL image from a dataset example."""
    img = example[image_field]
    if isinstance(img, dict) and "bytes" in img:
        from io import BytesIO
        return Image.open(BytesIO(img["bytes"])).convert("RGB")
    return img.convert("RGB")

def build_chat_messages(example, image_field="image", species_field="species"):
    """Build a chat-format prompt for a single example."""
    image = get_image(example, image_field)
    species = example[species_field]
    messages = [
        {
            "role": "user",
            "content": [
                {"type": "image", "image": image},
                {"type": "text", "text": "What species is shown in this image? Answer with the species name only."},
            ],
        },
        {
            "role": "assistant",
            "content": [{"type": "text", "text": species}],
        },
    ]
    return messages

def build_prompt_messages(example, image_field="image"):
    """Build a chat-format prompt without the assistant answer (for generation)."""
    image = get_image(example, image_field)
    messages = [
        {
            "role": "user",
            "content": [
                {"type": "image", "image": image},
                {"type": "text", "text": "What species is shown in this image? Answer with the species name only."},
            ],
        },
    ]
    return messages

print("Utility functions defined.")
```

```
Utility functions defined.
```

```
def generate_one(model, processor, example, image_field="image", max_new_tokens=128):
    """Run inference on a single example and return the decoded output."""
    messages = build_prompt_messages(example, image_field)
    text = processor.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
    image = get_image(example, image_field)
    inputs = processor(text=[text], images=[image], return_tensors="pt").to(model.device)
    with torch.no_grad():
        output_ids = model.generate(**inputs, max_new_tokens=max_new_tokens)
    # Decode only the newly generated tokens
    generated = output_ids[0][inputs["input_ids"].shape[1]:]
    return processor.decode(generated, skip_segment_tokens=True).strip()

print("Generation function defined.")
```

```
Generation function defined.
```

Mathematical setup for the common VLM policy

Before introducing methods, let's fix the model once in terms that all later sections can reuse.

A modern autoregressive vision-language model can be written as a conditional distribution

$$p_\theta(y \mid x, c) = \prod_{t=1}^{|y|} p_\theta(y_t \mid x, c, y_{<t})$$

Variable meanings:

- x is the text prompt, instruction, or textual context.
- c is the visual input (for one image or multiple images, or video frames).
- $y = (y_1, y_2, \dots, y_{|y|})$ is the target answer, written as a sequence of tokens.
- y_t is the token at position t.
- $y_{<t}$ is all previous answer tokens before position t.
- θ is the model's weights.
- $p_\theta(y_t \mid x, c, y_{<t})$ is the probability the model assigns to the next token y_t given the prompt, the image(s), and the answer prefix so far.

The full probability $p_\theta(y \mid x, c)$ is the product of all these next-token probabilities.

In practice, we usually train on the **negative log-likelihood** (also called **cross-entropy loss**):

$$\mathcal{L}_{\text{SFT}}(\theta) = -\log p_\theta(y \mid x, c) = -\sum_{t=1}^{|y|} \log p_\theta(y_t \mid x, c, y_{<t})$$

Minimizing this loss means increasing the probability the model assigns to the correct answer tokens.

What changes across methods?

- **Methods 1 and 2** keep θ fixed and modify the context c.
- **Method 3** also keeps θ fixed, but changes c by retrieving relevant evidence.
- **Methods 4 through 7** update some or all of θ during training.

Throughout this lecture, the core question is always the same: how does a method change θ, c, or both?

Method — Zero-shot prompting

Show me

Shared across all runs: same city, same day, same prompt, same model, same sampling settings

Idea

In zero-shot prompting we do **not** change the model parameters and we do **not** add retrieved external knowledge. We present the pretrained model with an image and an instruction and ask it to solve the task directly.

For this lecture, the task is new species recognition and structured reporting. The model sees an organism image and is asked to produce a species prediction together with a structured description.

This is the cheapest point on the adaptation ladder:

- no gradient updates,
- no new labels,
- no task-specific training set,
- no vector index.

It is therefore the baseline for every later method.

Mathematical view

Let the text instruction be x_t, the image be x_v, and let the generated answer be $y = (y_1, \dots, y_T)$. The full definition is

$$p(y \mid x_t, x_v) = \prod_{t=1}^{T} p_\theta(y_t \mid x_t, x_v, y_{<t})$$

Variable meanings:

- θ: pretrained model parameters
- x_t: the image
- y: the whole generated answer
- y_t: the token generated at step t
- $y_{<t}$: the prefix already generated before step t
- T: the number of generated tokens
- p_θ: the probability model parameterized by θ

A deterministic decoding rule then approximates the most likely answer:

$$\hat{y} \approx \arg\max_{y} p(y \mid x_t, x_v)$$

The important point is that the parameters θ are fixed during adaptation. **We do not run an optimizer.**

So zero-shot adaptation changes the **question** being asked, not the model.

Why does it work?

Zero-shot works because the pretrained VLM has already learned broad visual and textual regularities during large-scale pretraining and instruction tuning. If the current task is close to the kind of question it was trained to answer, a well-written prompt may be enough.

But zero-shot is limited by what the model already knows. If current species have real-world shifts that are absent from the pretrained data, no prompt can fully repair that.

Code Summary

The next cells do three things:

1. build a single multimodal prompt for one species-card request,
2. run the model in zero-shot mode,
3. evaluate the same prompt on a small batch of cases.

That gives us a clean baseline before we move to stronger but more expensive methods.

```
# Zero-shot prompting: one image + one instruction, no examples, no retrieval.
# Assumes `model`, `processor`, `device`, `torch`, `Image`, `requests`, `BytesIO`, `pd`, `time`, `json`, `re`
# are already defined in the notebook.

DEFAULT_IMAGE_URL = "https://upload.wikimedia.org/wikipedia/commons/thumb/3/3a/Cat03.jpg/1200px-Cat03.jpg"

def load_image(image_ref: str):
    """Load an image from a URL or a local path."""
    if image_ref.startswith("http://") or image_ref.startswith("https://"):
        response = requests.get(image_ref, timeout=30)
        response.raise_for_status()
        return Image.open(BytesIO(response.content)).convert("RGB")
    return Image.open(image_ref).convert("RGB")

def build_zero_shot_prompt(task_text: str) -> str:
    """Wrap the task in a minimal chat-style prompt."""
    return (
        "<|im_start|>user\n"
        "<image>\n"
        f"{task_text}\n"
        "<|im_end|>\n"
        "<|im_start|>assistant\n"
    )

def run_zero_shot(image_ref: str, task_text: str, max_new_tokens: int = 256) -> str:
    """Run one zero-shot request and return only the generated answer."""
    image = load_image(image_ref)
    prompt = build_zero_shot_prompt(task_text)

    inputs = processor(text=prompt, images=image, return_tensors="pt")
    inputs = {k: v.to(device) for k, v in inputs.items()}

    with torch.no_grad():
        output_ids = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
        )

    # Decode only the newly generated tokens, not the prompt.
    prompt_len = inputs["input_ids"].shape[1]
    answer = processor.tokenizer.decode(
        output_ids[0][prompt_len:], skip_segment_special_tokens=True
    )
    return answer.strip()

# Example: one species-card request.
task = (
    "Identify the species in this image. "
    "Return a species card with: common name, scientific name, "
    "habitat, diet, and one conservation status."
)
print(run_zero_shot(DEFAULT_IMAGE_URL, task))
```

```
# Batch evaluation: run the same prompt on a small set of cases.
# Assumes `run_zero_shot` and `pd` are already defined.

test_cases = [
    {"image_ref": "https://upload.wikimedia.org/wikipedia/commons/thumb/3/3a/Cat03.jpg/1200px-Cat03.jpg",
     "task": "What species is shown? Answer briefly."},
    {"image_ref": "https://upload.wikimedia.org/wikipedia/commons/thumb/3/3a/Cat03.jpg/1200px-Cat03.jpg",
     "task": "Describe the habitat and diet of this animal."},
]

rows = []
for case in test_cases:
    answer = run_zero_shot(case["image_ref"], case["task"])
    rows.append({
        "image_ref": case["image_ref"],
        "task": case["task"],
        "answer": answer,
    })

results = pd.DataFrame(rows)
results
```

Issues with Zero-Shot

Prompt sensitivity

Small wording changes can change the output. The model can change confidently when we alter wording, output format, or the amount of detail guidance in the prompt.

No new knowledge

If the model does not already know the species or its conservation status, zero-shot prompting cannot learn them.

Weak calibration

The model may produce a confident name confidently even when the image is ambiguous or when the species has outside the working subset.

Fine-grained recognition is hard

Species often differ in small visual details. Zero-shot can miss those cues and still produce a fluent, misleading, or context-poor answer.

Key references for Zero-Shot Learning

- Brown et al., Language Models are Few-Shot Learners (2020)
- Radford et al., Learning Transferable Visual Models From Natural Language Supervision (2021)
- Alayrac et al., Flamingo: a Visual Language Model for Few-Shot Learning (2022)

Method - Few-shot / in-context learning

Construction: system prompt + curated support examples + query, all packed into one multi-turn context window. Inference: the model predicts — all learning is in-context.

Idea

Now what prompting keeps the **model weights frozen**, but it adds a small number of **worked examples inside the context window**. These examples show the model what kind of answer we want.

For our species task, the full solve examples demonstrate the desired structured output:

- scientific name,
- common name,
- habitat,
- diet,
- genus,
- conservation status.

The goal is **not** to change the parameters by gradient descent. The goal is to change the conditioning information that enters the forward pass.

Mathematical view

Let the demonstrations set $\mathcal{D}$ be ordered as the prompt be

$$\mathcal{D}_m = (x_1, y_1, x_2, y_2, \dots, x_m, y_m)$$

Here:

- m is the number of demonstrations,
- x_i is the instruction text in demonstration i,
- y_i is the answer text in demonstration i,
- x_q is the current query in demonstration i.

For a new query x_q, the model defines

$$p(y \mid x_q) = p_\theta(y \mid \mathcal{D}_m, x_q)$$

The autoregressive factorization is still

$$p_\theta(y \mid \mathcal{D}_m, x_q) = \prod_{t=1}^{T} p_\theta(y_t \mid \mathcal{D}_m, x_q, y_{<t})$$

Here:

- $y = (y_1, \dots, y_T)$ is the answer, tokenized for the text server,
- y_t is the token generated at step t,
- $y_{<t}$ is the prefix already generated,
- θ is still the original frozen pretrained parameter set.

There is **no** optimizer update such as

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$$

Why can few-shot learning work if it does not change?

This is the central conceptual question.

The short answer is: **the context changes even though the weights do not.**

A transformer computes hidden states by attending over the full prompt. We can count demonstrations, we change:

- the tokens that are present,
- the attention pattern available to later tokens,
- the relative scores between query tokens and demonstration tokens,
- and therefore the hidden state trajectory that leads to the output logits.

So the model can behave differently on the query because the prompt has made a different computation easy.

It useful way to see this mathematically is

$$h_t = f_\theta(x_t, \mathcal{D}_m, y_{<t})$$

where h_t is the hidden representation used to predict token y_t. Since $\mathcal{D}_m$ changes, the hidden state h_t changes. That changes the logits, and therefore the final answer.

Why does it potentially work?

This next prompting can work for at least four reasons.

Format induction

It tells the model what kind of answer we want. The model infers that the new answer should follow that same schema.

Task disambiguation

A short user prompt may be ambiguous. Demonstrations tell the model what we mean by "solve" in this context.

For example, "identify the species" could mean:

- give a common name,
- give a scientific name,
- provide a description,
- provide a conservation status.

A few examples resolve this ambiguity.

Label-space restriction

When the demonstrations show the same candidate set and style, the model is nudged toward a specific subset over that candidate space.

Implicit Bayesian-style adaptation

One view from the literature of the model as using demonstrations to infer a latent task description and then solving the new query under that inferred task. For prediction decision purpose, but the framework view defines the in-memory task adaptation.

What few-shot learning is not

Few-shot prompting is not:

- gradient-based learning of the new species,
- systematic storage of those examples in the weights,
- a reliable way to inject many facts that are absent from pretraining.

Once the prompt disappears, the adaptation disappears.

Why can it fail?

This can be misleading as the examples:

- the demonstrations are poorly chosen,
- the examples are too similar to the current window,
- the model needs more facts than the prompt can provide,
- it needs visual distinctions that the examples do not make explicit.

A few examples cannot replace training.

Label-space restriction

When the examples always have the same candidate set and style, the model is nudged toward a specific subset over that candidate space.

Implicit Bayesian-style adaptation

One view from the literature of the model as using demonstrations to infer a latent task description and then solving the new query under that inferred task. For prediction decision purpose, but the framework view defines the in-memory task adaptation.

What few-shot learning is not

Few-shot prompting is not:

- gradient-based learning of the new species,
- systematic storage of those examples in the weights,
- a reliable way to inject many facts that are absent from pretraining.

Once the prompt disappears, the adaptation disappears.

Why can it fail?

This can be misleading as the examples:

- the demonstrations are poorly chosen,
- the examples are too similar to the current window,
- the model needs more facts than the prompt can provide,
- it needs visual distinctions that the examples do not make explicit.

A few examples cannot replace training.

Code Summary

The code below changes nothing from the prompt path, inserts them into the prompt, and queries the same black VLM. The only thing that changes relative to zero-shot prompting is the context.

Issues

- It is very **expensive**: every call now processes several extra images and many extra tokens.
- It is **unstable**: example order and example choice can flip results.
- It can **fail** if the prompt becomes too long or too confusing.

References for Few-Shot learning

- Brown, T. B., Mann, B., Ryder, N., et al. Language Models are Few-Shot Learners. (2020)
- Min, S., Lyu, X., Holtzman, A., et al. Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? (2022)
- Alayrac, J.-B., Donahue, J., Luc, P., et al. Flamingo: a Visual Language Model for Few-Shot Learning. (2022)
- Liu, X. G., Chen, F. H., Shao, J., Yang, M., Kuo, C. J. ASAP: Attention-Shift-Aware Pruning for Efficient LVLM Inference and Fine-tuning. (2026)

```
# Few-shot prompting: same image + same task, but with examples in the context.
# Assumes `model`, `processor`, `device`, `torch`, `Image`, `requests`, `BytesIO`, `pd`, `time`, `json`, `re`
# are already defined in the notebook.

DEFAULT_IMAGE_URL = "https://upload.wikimedia.org/wikipedia/commons/thumb/3/3a/Cat03.jpg/1200px-Cat03.jpg"

def load_image(image_ref: str):
    """Load an image from a URL or a local path."""
    if image_ref.startswith("http://") or image_ref.startswith("https://"):
        response = requests.get(image_ref, timeout=30)
        response.raise_for_status()
        return Image.open(BytesIO(response.content)).convert("RGB")
    return Image.open(image_ref).convert("RGB")

# Each example is a (image_ref, task_text, answer_text) triple.
# Keep the answer format identical to what we want the model to produce.
FEW_SHOT_EXAMPLES = [
    {
        "image_ref": "https://upload.wikimedia.org/wikipedia/commons/thumb/3/3a/Cat03.jpg/1200px-Cat03.jpg",
        "task": "Identify the species in this image. Return a species card with: common name, scientific name, habitat, diet, and one conservation status.",
        "answer": (
            "common name: domestic cat\n"
            "scientific name: Felis catus\n"
            "habitat: human settlements, farms, and urban areas worldwide\n"
            "diet: carnivore; small mammals, birds, insects\n"
            "conservation status: not evaluated (domesticated)"
        ),
    },
    # Add more examples here. Aim for 2–4 that cover different species.
]

def build_few_shot_prompt(task_text: str) -> str:
    """Wrap examples + the new task in a chat-style prompt."""
    parts = []
    for ex in FEW_SHOT_EXAMPLES:
        parts.append(
            "<|im_start|>user\n"
            "<image>\n"
            f"{ex['task']}\n"
            "<|im_end|>\n"
            "<|im_start|>assistant\n"
            f"{ex['answer']}\n"
            "<|im_end|>\n"
        )
    parts.append(
        "<|im_start|>user\n"
        "<image>\n"
        f"{task_text}\n"
        "<|im_end|>\n"
        "<|im_start|>assistant\n"
    )
    return "".join(parts)

def run_few_shot(image_ref: str, task_text: str, max_new_tokens: int = 256) -> str:
    """Run one few-shot request and return only the generated answer."""
    image = load_image(image_ref)
    prompt = build_few_shot_prompt(task_text)

    inputs = processor(text=prompt, images=image, return_tensors="pt")
    inputs = {k: v.to(device) for k, v in inputs.items()}

    with torch.no_grad():
        output_ids = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
        )

    prompt_len = inputs["input_ids"].shape[1]
    answer = processor.tokenizer.decode(
        output_ids[0][prompt_len:], skip_segment_special_tokens=True
    )
    return answer.strip()

# Example: same query as the zero-shot baseline, but with examples in front.
task = (
    "Identify the species in this image. "
    "Return a species card with: common name, scientific name, "
    "habitat, diet, and one conservation status."
)
print(run_few_shot(DEFAULT_IMAGE_URL, task))
```

```
# Batch evaluation: run the few-shot prompt on a small set of cases.
# Assumes `run_few_shot` and `pd` are already defined.

test_cases = [
    {"image_ref": "https://upload.wikimedia.org/wikipedia/commons/thumb/3/3a/Cat03.jpg/1200px-Cat03.jpg",
     "task": "What species is shown? Answer briefly."},
    {"image_ref": "https://upload.wikimedia.org/wikipedia/commons/thumb/3/3a/Cat03.jpg/1200px-Cat03.jpg",
     "task": "Describe the habitat and diet of this animal."},
]

rows = []
for case in test_cases:
    answer = run_few_shot(case["image_ref"], case["task"])
    rows.append({
        "image_ref": case["image_ref"],
        "task": case["task"],
        "answer": answer,
    })

results = pd.DataFrame(rows)
results
```

Issues With Few-Shot Learning

Context window cost

Each demonstration **consumes tokens**, and multimodal examples are **expensive** in both memory and latency.

Example selection matters

Even a **semiparametric** demonstrations can hurt performance. In few-shot species tasks, support examples should ideally resemble the target class and label granularity.

No persistent learning

Once the context is removed, the adaptation disappears. Thus short prompting does not produce a durable trained model.

Susceptibility to ordering effects

The order of examples can matter. This is a reminder that in-context learning is still an inference-time phenomenon rather than an explicit optimization-based learning procedure.

Key references

- Brown et al., Language Models are Few-Shot Learners (2020)
- Min et al., Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? (2022)
- Alayrac et al., Flamingo: a Visual Language Model for Few-Shot Learning (2022)

Method - Retrieval-Augmented Generation (RAG)

Simple RAG

RAG with Backpropagation

Idea

RAG augments the model with an **external memory**. Instead of asking only on pretrained parameters or only on in-context demonstrations, we retrieve relevant documents and place them into the generation context.

For the new species task, the retrieved documents are **species cards** derived from our catalogue. They can contain:

- scientific and common name,
- habitat and diet,
- morphological traits,
- conservation status,
- distinguishing features.

Conceptually, RAG separates two roles:

- the **retriever** decides which evidence to bring in,
- the **generator** decides how to answer using that evidence.

Basic probabilistic picture

Let:

- x be the text instruction,
- z be the image,
- d be a latent document/chunk produced from the instruction and image,
- y be the answer/response,
- $p_\eta(d \mid x, z)$ be the retriever,
- $p_\theta(y \mid x, z, d)$ be the generator,
- $\mathcal{D}_k = \{d_1, \dots, d_k\}$ be the top-k documents.

A standard latent-document RAG model writes

$$p_{\eta,\theta}(y \mid x, z) = \sum_{d \in \mathcal{D}_k} p_\eta(d \mid x, z)\, p_\theta(y \mid x, z, d)$$

This equation means:

- the retriever assigns a probability to each document through $p_\eta(d \mid x, z)$,
- the generator assigns an answer likelihood conditioned on one document through $p_\theta(y \mid x, z, d)$,
- we marginalize over the retrieved documents.

In practice, we do not sum over the full corpus but only over the retrieved set $\mathcal{D}_k$.

$$p_{\eta,\theta}(y \mid x, z) \approx \sum_{d \in \mathcal{D}_k} p_\eta(d \mid x, z)\, p_\theta(y \mid x, z, d)$$

The generator itself remains autoregressive:

$$p(y \mid x, z, d) = \prod_{t=1}^{T} p_\theta(y_t \mid x, z, d, y_{<t})$$

Here:

- T is the length of the answer,
- y_t is the t-th token,
- $y_{<t}$ is the prefix before step t,
- θ is the decoder weight.

3. What is the gold target in RAG?

A training fact. In supervised RAG, the target is the ground-truth answer sequence:

$$y^\star = (y_1^\star, \dots, y_T^\star)$$

For our fieldwork, $y^\star$ is the structured note species answer we want the model to produce, such as a short scientific name, family, genus, and a short evidence sentence.

We also assume $y^\star$ becomes RAG training time to retrieve.

Let each particular such fixed answer. The gold target tells the system which retrieved evidence was actually useful for producing the desired answer.

A retrieval label. Some RAG setups also have a retrieval label, for example a relevance judgment for a query-document pair.

Marginal log-likelihood and why the equations jump

The training objective is to maximize the log-likelihood of the gold answer, marginalized over retrieved documents:

$$\mathcal{L}_{\text{RAG}}(\theta, \eta) = \log p_{\eta,\theta}(y^\star \mid x, z)$$

Expanding the latent document expression, the objective is

$$\mathcal{L}_{\text{RAG}}(\theta, \eta) = \log \sum_{d \in \mathcal{D}_k} p_\eta(d \mid x, z)\, p_\theta(y^\star \mid x, z, d)$$

So the "what" is simply this:

1. define the answer probability as a latent variable mixture over documents,
2. score the gold answer under each document,
3. train the retriever and generator jointly.

Nothing mysterious is happening. It is just maximum likelihood with a latent retrieval decision.

Posterior responsibility weights $w(d)$

A very useful quantity is

$$w(d) = \frac{p_\eta(d \mid x, z)\, p_\theta(y^\star \mid x, z, d)}{\sum_{d' \in \mathcal{D}_k} p_\eta(d' \mid x, z)\, p_\theta(y^\star \mid x, z, d')}$$

This is a common responsibility weight.

Here:

- the numerator says how plausible document d was before seeing the target and how well it explains the target,
- the denominator normalizes across all documents in the candidate set.

So $w(d)$ answers the question:

> **After seeing the gold answer, how much responsibility should document d receive for explaining it?**

Gradient with respect to the generator

If a document was retrieved and it was useful for the gold answer, the weight $w(d)$ is large.

If a document was retrieved but did not help explain the gold answer, $w(d)$ is small.

This is why $w(d)$ is so important. It tells us which documents were actually useful, so we can direct updates more strongly toward them.

Gradient with respect to the generator

Start from

$$\nabla_\theta \log p_{\eta,\theta}(y^\star \mid x, z) = \nabla_\theta \log \sum_{d \in \mathcal{D}_k} p_\eta(d \mid x, z)\, p_\theta(y^\star \mid x, z, d)$$

Differentiate with respect to θ:

$$\nabla_\theta \log p_{\eta,\theta}(y^\star \mid x, z) = \sum_{d \in \mathcal{D}_k} w(d)\, \nabla_\theta \log p_\theta(y^\star \mid x, z, d)$$

This says the generator update is a responsibility-weighted average of per-document gradient updates.

Interpretation:

- if a document strongly supports the gold answer, it gets a large weight,
- the model learns more strongly from that document's conditional path.

Gradient with respect to the retriever

Similarly, the retriever gradient is

$$\nabla_\eta \log p_{\eta,\theta}(y^\star \mid x, z) = \sum_{d \in \mathcal{D}_k} w(d)\, \nabla_\eta \log p_\eta(d \mid x, z)$$

So the retriever is pushed to assign more mass to documents that, in hindsight, explain the gold answer well.

This is the standard way to understand end-to-end differentiable RAG.

- the generator learns to use retrieved evidence,
- the retriever learns to find documents that make it support the gold target.

Why this matters even if we use a frozen retriever

In our example implementation, we use a lightweight TF-IDF retriever and do not backpropagate through retrieval. That is common in real systems because it is simpler and more stable.

But the mental model we still have is important. It explains why retrieval can help even without training the retriever.

- it can provide evidence that the model does not memorize,
- it can differentiate RAG from a basic cache of generated answers.

Token-level intuition

Another way to understand RAG is at the token level.

Without retrieval, the next-token distribution is

$$p(y_t \mid x, z, y_{<t})$$

With retrieval, the model instead uses

$$\sum_{d \in \mathcal{D}_k} p_\eta(d \mid x, z)\, p_\theta(y_t \mid x, z, d, y_{<t})$$

So retrieval changes the logits indirectly by changing the context used to compute them.

Code Summary

The code below combines multiple parts from the last notebooks, builds a simple vector space retrieval, simulates how IR works for each query, and appends those results to the original spreadsheet.

Issues

- Retrieval quality matters as much as generation
- Chunking strategy can make or break retrieval
- In a two-hour MVP, the retriever does not have time documentation to debug
- Technical jargon increases tasks (unfiltered) if the retrieval evidence is wrong or sparse

References

- Lewis, P., Perez, E., Piktus, A., et al.
  [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](https://arxiv.org/abs/2005.11401)
- Karpukhin, V., Oguz, B., Min, S., et al.
  [Dense Passage Retrieval for Open-Domain Question Answering](https://arxiv.org/abs/2004.04906)
- Gao, L., Ma, X., Lin, J., & Callan, J.
  [Precise Zero-Shot Dense Retrieval without Relevance Labels](https://arxiv.org/abs/2212.10496)
- Ma, X., Fang, L., Zhang, N., Zhang, X.
  [A Thorough Examination on Zero-shot Dense Retrieval](https://arxiv.org/abs/2204.12755)
- Ma, X., Yu, C., Zhang, X., et al.
  [Fine-tuning LLaMA for Multi-stage Text Retrieval](https://arxiv.org/abs/2310.08319)

```
# ── Install dependencies ──────────────────────────────────────────────────────
# pip install sentence-transformers faiss-cpu openpyxl pandas numpy

import os
import re
import pickle
import numpy as np
import pandas as pd
import faiss
from sentence_transformers import SentenceTransformer

# ── Configuration ─────────────────────────────────────────────────────────────
EXCEL_PATH = "Copy of Sample Data for Exercises.xlsx"
SHEET_NAME = "Sample Data for Exercises"
QUERY_COL  = "Search Navigation Query"
OUTPUT_COL = "Retrieved Context"
INDEX_PATH = "faiss_index.pkl"
TOP_K      = 3

# ── Load data ─────────────────────────────────────────────────────────────────
df = pd.read_excel(EXCEL_PATH, sheet_name=SHEET_NAME)
print(f"Loaded {len(df)} rows")

# ── Build or load FAISS index ─────────────────────────────────────────────────
model = SentenceTransformer("all-MiniLM-L6-v2")

if os.path.exists(INDEX_PATH):
    print("Loading existing index...")
    with open(INDEX_PATH, "rb") as f:
        index, chunks = pickle.load(f)
else:
    print("Building index...")
    chunks = df["Content"].fillna("").tolist()
    embeddings = model.encode(chunks, show_segment_progress=True)
    embeddings = np.array(embeddings).astype("float32")
    index = faiss.IndexFlatL2(embeddings.shape[1])
    index.add(embeddings)
    with open(INDEX_PATH, "wb") as f:
        pickle.dump((index, chunks), f)
    print(f"Index saved to {INDEX_PATH}")

# ── Retrieve top-k chunks for each query ──────────────────────────────────────
queries = df[QUERY_COL].fillna("").tolist()
query_embeddings = model.encode(queries, show_segment_progress=True)
query_embeddings = np.array(query_embeddings).astype("float32")

_, indices = index.search(query_embeddings, TOP_K)

df[OUTPUT_COL] = [
    "\n\n---\n\n".join(chunks[i] for i in row_indices)
    for row_indices in indices
]

# ── Save back to Excel ────────────────────────────────────────────────────────
df.to_excel(EXCEL_PATH, sheet_name=SHEET_NAME, index=False)
print(f"Done. Retrieved context written to '{OUTPUT_COL}' column.")
```

```
Loaded 1000 rows
Building index...
Batches: 100%|██████████| 32/32 [00:04<00:00,  7.12it/s]
Index saved to faiss_index.pkl
Batches: 100%|██████████| 1/1 [00:00<00:00, 18.43it/s]
Done. Retrieved context written to 'Retrieved Context' column.
```

Issues

- Retrieval quality is a bottleneck
- If the correct opening exists in the knowledge base, the generator cannot see it

Retrieval errors can be amplified

- A confident but wrong answer is worse than no answer

Frozen retrieval is not end-to-end training

- In practice (with ~2 TF-IDF-based systems), the retrieval stage is often non-differentiable and fixed. That makes engineering easier, but it also means the retriever itself does not improve during multimodal training.

Long contexts create cost

- As new articles have specific rules, latency and memory use, and the model may become distracted by irrelevant evidence.

```
# ── Install dependencies ──────────────────────────────────────────────────────
# pip install sentence-transformers faiss-cpu openpyxl pandas numpy anthropic

import os
import re
import pickle
import numpy as np
import pandas as pd
import faiss
import anthropic
from sentence_transformers import SentenceTransformer

# ── Configuration ─────────────────────────────────────────────────────────────
EXCEL_PATH = "Copy of Sample Data for Exercises.xlsx"
SHEET_NAME = "Sample Data for Exercises"
QUERY_COL  = "Search Navigation Query"
OUTPUT_COL = "Retrieved Context"
INDEX_PATH = "faiss_index.pkl"
TOP_K      = 3

# ── Load data ─────────────────────────────────────────────────────────────────
df = pd.read_excel(EXCEL_PATH, sheet_name=SHEET_NAME)
print(f"Loaded {len(df)} rows")

# ── Build or load FAISS index ─────────────────────────────────────────────────
model = SentenceTransformer("all-MiniLM-L6-v2")

if os.path.exists(INDEX_PATH):
    print("Loading existing index...")
    with open(INDEX_PATH, "rb") as f:
        index, chunks = pickle.load(f)
else:
    print("Building index...")
    chunks = df["Content"].fillna("").tolist()
    embeddings = model.encode(chunks, show_segment_progress=True)
    embeddings = np.array(embeddings).astype("float32")
    index = faiss.IndexFlatL2(embeddings.shape[1])
    index.add(embeddings)
    with open(INDEX_PATH, "wb") as f:
        pickle.dump((index, chunks), f)
    print(f"Index saved to {INDEX_PATH}")

# ── Retrieve top-k chunks for each query ──────────────────────────────────────
queries = df[QUERY_COL].fillna("").tolist()
query_embeddings = model.encode(queries, show_segment_progress=True)
query_embeddings = np.array(query_embeddings).astype("float32")

_, indices = index.search(query_embeddings, TOP_K)

df[OUTPUT_COL] = [
    "\n\n---\n\n".join(chunks[i] for i in row_indices)
    for row_indices in indices
]

# ── Generate answers with Claude ──────────────────────────────────────────────
client = anthropic.Anthropic()  # reads ANTHROPIC_API_KEY from environment

def generate_answer(query, context):
    response = client.messages.create(
        model="claude-haiku-4-5-20251001",
        max_tokens=256,
        system="You are a helpful assistant. Answer the user's question using only the provided context. If the context does not contain enough information, say so.",
        messages=[{"role": "user", "content": f"Context:\n{context}\n\nQuestion: {query}"}],
    )
    return response.content[0].text

df["Generated Answer"] = [
    generate_answer(q, c)
    for q, c in zip(queries, df[OUTPUT_COL])
]

# ── Save back to Excel ────────────────────────────────────────────────────────
df.to_excel(EXCEL_PATH, sheet_name=SHEET_NAME, index=False)
print(f"Done. Retrieved context and generated answers written to Excel.")
```

```
Loaded 1000 rows
Loading existing index...
Batches: 100%|██████████| 1/1 [00:00<00:00, 21.07it/s]
Done. Retrieved context and generated answers written to Excel.
```

Issues

- Retrieval can be good enough to look grounded, but still miss the key detail
- Chunking can split important information across boundaries
- Top-k retrieval may include duplicate or near-duplicate passages, wasting context window
- Retrieved text may contain boilerplate, navigation clutter, or irrelevant sections
- Long contexts increase latency and cost
- Models may ignore relevant evidence buried in the middle of a long prompt
- Citation quality can look better than it is if the model paraphrases loosely

Fixes

- Use smaller, more focused chunks
- Add overlap between chunks
- Deduplicate retrieved passages before generation
- Rerank retrieved chunks before sending them to the model
- Limit the number of chunks unless they add clear value
- Require the model to answer only from retrieved context
- Add a fallback response when evidence is weak or missing
- Track retrieval and generation errors separately

Issues with RAG

- Retrieval quality is a bottleneck
- If the correct opening exists in the knowledge base, the generator cannot see it

Retrieval errors can be amplified

- A confident but wrong answer is worse than no answer

Frozen retrieval is not end-to-end training

- In practice (with ~2 TF-IDF-based systems), the retrieval stage is often non-differentiable and fixed. That makes engineering easier, but it also means the retriever itself does not improve during multimodal training.

Long contexts create cost

- As new articles have specific rules, latency and memory use, and the model may become distracted by irrelevant evidence.

Key references

- Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (2020)
- Karpukhin et al., Dense Passage Retrieval for Open-Domain Question Answering (2020)
- Gao et al., Precise Zero-Shot Dense Retrieval without Relevance Labels (2022)

Method — Supervised fine-tuning (SFT)

Idea

Supervised fine-tuning moves from inference to the adaptation. We now provide labeled image-instruction-answer triples and update the model so that its observed answer becomes more likely not only in this one prompt, but across the task distribution.

For the new-service task, each training example contains:

- an image,
- an instruction asking for structured service identification,
- and the target JSON answer we want the model to produce.

This is the standard training pattern for turning a general VLM into a domain-specialist assistant.

Mathematical view

Let the supervised dataset be

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$$

Here,

- x_i is the number of training examples,
- x_i is the instruction for example i,
- y_i is the target answer for example i,
- $y_{i,t}$ is the target token at position t,
- $y_{i,<t}$ is the target length for example i,
- θ is the target length for example i.

The model learns how the target probability as

$$p_\theta(y_i \mid x_i) = \prod_{t=1}^{|y_i|} p_\theta(y_{i,t} \mid x_i, y_{i,<t})$$

The standard causal language model objective is the negative log likelihood

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{i=1}^{N} \log p_\theta(y_i \mid x_i)$$

If we use a hard mask $m_{i,t} \in \{0,1\}$ to decide which tokens count, then

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{i=1}^{N} \sum_{t=1}^{|y_i|} m_{i,t} \log p_\theta(y_{i,t} \mid x_i, y_{i,<t})$$

Here,

- $m_{i,t} = 1$ means token t contributes to the loss,
- $m_{i,t} = 0$ means token t is ignored.

$$\pi \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{A}|} \quad \mathcal{S} \times \mathcal{A} \mapsto [0,1]$$

Here:

- $\mathcal{S}$ is the input vocab of the task.
- $\mathcal{A}$ is the output vocab of the task.
- π is the LLM we're trying to train.
- $\mathbb{R}$ is the set of real numbers.
- $\mathbb{R}$ and $\mathcal{S}$ are hyperparams for code matching.
- $\mathbb{R}$ and $\mathcal{A}$ are free

The classic PPO VLA recipe is

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\hat{A}_t,\ \text{clip}\big(r_t(\theta),1-\epsilon,1+\epsilon\big)\hat{A}_t\big)\Big],\quad r_t(\theta)=\frac{\pi_\theta(a_t\mid s_t)}{\pi_{\theta_{\text{old}}}(a_t\mid s_t)}$$

Here:

- s_t is the combined observation.
- a_t is the tokenized robot action.
- $\hat{A}_t$ is the advantage.

Intuition:

- If the update would move the ratio too far from the old policy, the clipped term stops the incentive from growing.
- In VLA training this is a form of trust-region approximation.

How this relates to a VLM

In a VLA:

- s_t is the image + instruction + robot state, and the previously generated tokens.
- The action a_t is the next token.
- The policy π_θ is the full model (vision encoder, LLM backbone, detokenizer, format, calibration, or human preferences).
- The entropy is a bonus term, also applied to action tokens.

So PPO for a VLM is just token-level reinforcement learning with a clipped objective.

Important practical note

For VLMs, the clearest efficiency discovered online RL path today is often not classical PPO but trainers such as RLOO or Online DPO. We will study PPO because it gives the conceptual template for the algorithms you actually use.

References

- Andrew, N., Kochenderfer, D., Singh, S., Moravec, T. [Policy Gradient Methods for Reinforcement Learning with Function Approximation](https://proceedings.neurips.cc/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf)
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O. [Proximal Policy Optimization Algorithms](https://arxiv.org/abs/1707.06347)
- Hugging Face. [TRL documentation](https://huggingface.co/docs/trl/index)
- [PPO Trainer](https://huggingface.co/docs/trl/ppo_trainer)

Mapping the RL notation back to a VLM

For language or vision-language generation, the RL objects correspond to:

- **state** s_t: the prompt, image, and generated prefix so far.
- **action** a_t: the next token.
- **policy** $\pi_\theta(a_t \mid s_t)$: the model's next-token distribution.
- **trajectory** τ: the full generated response.

The reward is usually given only at the end of the response, by scoring the whole completion.

The image is the key: it is constant for all T generated tokens, so PPO is performing token-level updates on a trajectory whose state always includes the same visual context.

Method — Preference tuning with DPO

Idea

DPO stands for **Direct Preference Optimization**. Instead of asking for reals to initialize a single goal answer, DPO asks the model to take a **preferred answer** above a **rejected answer**.

This is useful when there are many acceptable outputs but some are clearly better than others. In the raw reporter task, we may prefer answers that:

- name the species correctly,
- mention the habitat,
- state uncertainty when appropriate,
- avoid unsupported claims.

Preference data

For each prompt image pair (x, y) we construct:

- x is the prompt,
- y_w is the chosen answer,
- y_l is the rejected answer,
- π_θ is the model being trained,
- π_{ref} is a frozen reference model, usually the starting SFT model,
- β controls the sharpness of the preference update.

The chosen answer does not need to be the unique correct answer. It only needs to be better than the rejected one.

DPO loss

A standard DPO loss is

$$\mathcal{L}_{\text{DPO}}(\theta) = -\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)$$

If we define the preference margin

$$m = \beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}$$

then

$$\mathcal{L}_{\text{DPO}}(\theta) = -\log \sigma(m)$$

Intuition behind the loss

The quantity

$$\log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x)$$

asks:

> how much more does the current policy like the winner than the loser?

The reference term

$$\log \pi_{\text{ref}}(y_w \mid x) - \log \pi_{\text{ref}}(y_l \mid x)$$

asks the same question for the frozen reference model.

So DPO is really comparing two margins:

- the policy's current preference margin,
- the reference model's preference margin.

If the policy does not improve enough over the reference, the loss is large. If the policy clearly prefers the chosen answer more strongly than the reference does, the loss becomes small.

Why the sigmoid appears

The sigmoid function $\sigma(z) = \frac{1}{1 + e^{-z}}$ converts the preference margin into a probability. The resulting loss behaves like logistic regression on pairwise preferences.

So DPO can be viewed as:

- a supervised classification objective,
- over chosen-versus-rejected pairs,
- with a built-in KL anchor to the reference model.

Why is DPO often easier than full RLHF?

DPO avoids several moving parts that RLHF usually needs:

- explicit reward-model fitting during the policy update,
- sampling-intensive tuning loops,
- online rollouts,
- PPO-style tuning.

It is a common starting point for a small, supervised-style alignment pipeline over pairs of answers.

Code Summary

The code below scores a chosen and rejected answer, computes the DPO loss, runs one backward pass, and checks the same loss both with the DPO objective.

Issues

- DPO can overfit if **preference pairs** are too few or too similar.
- If the **chosen** and **rejected** answers are too easy to separate, the learned preference can be shallow.
- The reference model matters: a poor π_{ref} can anchor the policy in the wrong place.
- DPO is only as good as the preference data. If the labels are noisy, the model will learn that noise.

References

- Andrew, N., Moreno, A., Mitchell, E., et al. [Direct Preference Optimization: Your Language Model is Secretly a Reward Model](https://arxiv.org/abs/2305.18290)
- Ouyang, L., Wu, J., Jiang, X., et al. [Training language models to follow instructions with human feedback](https://arxiv.org/abs/2203.02155)
- Andrew, N., Rosenblatt, J., & Wasserstein, A. (2022). [Understanding Dataset Difficulty with $\mathcal{V}$-Usable Information](https://proceedings.mlr.press/v162/ethayarajh22a.html)
- Wang, S., Zhang, Y., Chen, H., et al. [Beyond Reward Hacking: Causal Rewards for Preference Alignment](https://arxiv.org/abs/2502.04284)

DPO vs. KTO

KTO is another alternative to DPO and is sometimes easier to implement.

	KTO (Kahneman-Tversky Opt.)	DPO (Direct Preference Opt.)
Data	Single annotated response $(x, y, \text{label} = \text{good/bad})$ no paired comparison needed	Paired preference responses (x, y_w, y_l) — chosen & rejected requires pairwise annotation
β = 0	Policy π_θ free to drift	Reference π_{ref} frozen SFT model	Policy π_θ free to drift	Reference π_{ref} frozen SFT model
β = 1	$\sigma(r_\theta) \approx \text{help } \mathbb{E}[r_\theta]$ single-sample reward ascent	$\sigma(r_\theta) \approx \text{help } \sigma(r_\theta(y_w) - r_\theta(y_l))$ pairwise reward ascent
β = ∞	$\sigma(r_\theta) \to \text{hurt } \mathbb{E}[r_\theta]$ single-sample reward descent	$\sigma(r_\theta) \to \text{hurt } \sigma(r_\theta(y_w) - r_\theta(y_l))$ pairwise reward descent
Data	annotation-level	reference-level

β = regularization strength

$\beta = 0$ → forward pass

$\beta = 1$ → implicit reward r_θ (β)

$\beta = \infty$ → construction

KT value function $\sigma(r_\theta - \text{KL})$ (human-aware loss)

$\sigma(r_\theta - \text{KL}) - \sigma(\text{KL})$

if label = good

$\sigma(\text{KL}) - \sigma(r_\theta - \text{KL})$

if label = bad

$\mathbb{E}_{x,y}$ asymmetric targets

Contrastive margin

$\text{margin} = \beta \cdot (\log \pi_\theta(y_w) - \log \pi_{\text{ref}}(y_w))$

$- \beta \cdot (\log \pi_\theta(y_l) - \log \pi_{\text{ref}}(y_l))$

$\sigma(\text{margin})$ pairwise gain + loss

$\mathbb{E}$ implicit reward r_θ

$\mathbb{E}$ = Forward pass

β = 0 → Policy π_θ free to drift

β = 1 → $\sigma(r_\theta - \text{KL}) - \sigma(\text{KL})$

β = ∞ → $\sigma(r_\theta - \text{KL})$ → Reference π_{ref} frozen SFT model

β = Regularization strength

β = Hyperparameter

β = 0 → $\sigma(r_\theta - \text{KL})$ → $\sigma(r_\theta(y_w) - r_\theta(y_l))$

β = ∞ → $\sigma(r_\theta - \text{KL})$ → $\sigma(\text{KL})$ → $\sigma(r_\theta - \text{KL})$

- Ke, G., Chen, T.-Y., He, Y., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. Advances in Neural Information Processing Systems, 30.
- Li, X., Wang, Y., Yu, Y. (2020). Interpretable Intrusion Detection for Large-Scale Language Models via Rule-Aware Dual-Early Revision.

RLOO: Idea and core math

RLOO stands for **REINFORCE Leave-One-Out**. It is an **RL-based alignment method** in which the model generates several candidate completions for the **same prompt**, each completion receives a reward, and the policy is updated to increase the probability of completions that score better than the others.

Idea

Suppose we sample the same prompt x and generate K completions:

$$
y_1, y_2, \dots, y_K \sim \pi_\theta(\cdot \mid x)
$$

where:

- y_i is the i-th sampled completion,
- π_θ is the model's parameters,
- K is the number of samples,
- x is the sampled completion.

Each completion receives a reward:

$$
r_i = R(y_i, x_i)
$$

where:

- $R(y_i, x_i)$ is the reward assigned to completion y_i for prompt x_i.

The key idea of **leave-one-out** is to compare each completion against the **average reward of the other completions for the same prompt**.

For completion i, the baseline is:

$$
b_i = \frac{1}{K-1} \sum_{j \neq i} r_j
$$

Then the advantage for completion i is:

$$
A_i = r_i - b_i
$$

Intuition

This means:

- A completion is **better than the others** when $A_i > 0$.
- A completion is **worse than the others** when $A_i < 0$.

In the toy case:

- Valid is the other answers generated for the same prompt, use this answer unusually good or unusually bad.

If one only 1 valid answer **more** and **no** valid completion, we have more chance to get a better answer.

Policy-gradient view

RLOO is based on a REINFORCE-style policy-gradient update. A simplified version is:

$$
\nabla_\theta \mathcal{L}_{\text{RLOO}} = -\frac{1}{K} \sum_{i=1}^{K} A_i \nabla_\theta \log \pi_\theta(y_i \mid x)
$$

where:

- $\nabla_\theta \mathcal{L}_{\text{RLOO}}$ is the learning signal,
- ∇_θ is the gradient with respect to the parameters,
- $\log \pi_\theta(y_i \mid x)$ is the log-probability the model assigns to completion y_i.

Interpretation of the loss

The loss has a very natural meaning:

- If $A_i > 0$, then the model is encouraged to **increase** the probability of y_i.
- If $A_i < 0$, then the model is encouraged to **decrease** the probability of y_i.

So RLOO pushes the model toward completions that **score better than the others** for the same prompt.

KL regularization

In practice, the reward objective usually adds a penalty that keeps the updated policy close to a reference model π_{ref}. A common reward form is:

$$
r_i = R(y_i, x_i) - \beta \, \mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})
$$

where:

- β is the regularization strength,
- β controls how strongly we penalize drifting away from the reference model,
- π_{ref} is usually the supervised fine-tuned (SFT) model.

This term helps keep the model from moving too far from the reference model.

Why RLOO is useful

RLOO is attractive because:

- It is simpler than heavier RL methods such as PPO.
- It does not need a separate **critic** model.
- It works naturally when we can score several completions for the same prompt.

In this example

For our toy case study, the prompt is a **charged-and-capacitive-disinfection** and the reward can be based on things such as:

- whether the output is valid JSON,
- whether the output format is complete,
- whether the explanation is grounded,
- whether the explanation is consistent and concise.

So RLOO can be summarized as:

> generate several answers for the same image, score them, compare each answer to the others, and then update the model to make the better answers more likely to be chosen.

```
# Cell 1
from trl import RewardConfig, RewardTrainer, RLOOConfig, RLOOTrainer

def generate_with_prompt_template(model, tokenizer, prompt, include_description=True):
    if include_description:
        full_prompt = f"""Describe the properties in the image from the coordinate data.
Coordinate data:
{prompt}
Now describe the properties in the image from the coordinate data.
Description:"""
    else:
        full_prompt = f"""Describe the properties in the image from the coordinate data.
Coordinate data:
{prompt}
Description:"""
    inputs = tokenizer(full_prompt, return_tensors="pt").to(model.device)
    outputs = model.generate(**inputs, max_new_tokens=512, do_sample=False)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

def make_reward_model_inputs(tokenizer, prompt, completion, include_description=True):
    text = generate_with_prompt_template(
        tokenizer,
        prompt,
        include_description=include_description,
    ) + completion
    return tokenizer(text, truncation=True, max_length=2048)

reward_model_path = "./reward_model_qwen3_description_completion_only"
# reward_model_path = "./reward_model_qwen3_description_only"

reward_model = AutoModelForSequenceClassification.from_pretrained(
    reward_model_path,
    num_labels=1,
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
reward_tokenizer = AutoTokenizer.from_pretrained(reward_model_path)
if reward_tokenizer.pad_token is None:
    reward_tokenizer.pad_token = reward_tokenizer.eos_token
reward_model.config.pad_token_id = reward_tokenizer.pad_token_id
reward_model.eval()

def reward_fn(prompts, completions, **kwargs):
    rewards = []
    for prompt, completion in zip(prompts, completions):
        inputs = make_reward_model_inputs(
            reward_tokenizer,
            prompt,
            completion,
            include_description=True,
            # include_description=False,
        ).to(reward_model.device)
        with torch.no_grad():
            score = reward_model(**inputs).logits[0, 0].item()
        rewards.append(score)
    return rewards

rloo_config = RLOOConfig(
    output_dir="./rloo_qwen3_description_completion_only",
    # output_dir="./rloo_qwen3_description_only",
    learning_rate=1e-6,
    per_device_train_batch_size=1,
    gradient_accumulation_steps=8,
    num_generations=4,
    max_completion_length=512,
    num_train_epochs=1,
    logging_steps=10,
    save_steps=200,
    bf16=True,
    report_to="none",
)

rloo_trainer = RLOOTrainer(
    model=model,
    reward_funcs=reward_fn,
    args=rloo_config,
    train_dataset=dataset,
    processing_class=tokenizer,
)

rloo_trainer.train()
rloo_trainer.save_model("./rloo_qwen3_description_completion_only")
# rloo_trainer.save_model("./rloo_qwen3_description_only")
```

```
# Cell 2
import os
import torch
import pandas as pd
from datasets import Dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    DataCollatorWithPadding,
)

# ============================================================
# 1. Load CSV
# ============================================================
csv_path = "./reward_data_description_completion_only.csv"
# csv_path = "./reward_data_description_only.csv"
df = pd.read_csv(csv_path)

print("Loaded rows:", len(df))
print("Columns:", df.columns.tolist())

# ============================================================
# 2. Build prompt + completion text
# ============================================================
# Prompt = the same instruction + coordinate data used during RLOO generation
# Completion = the model's description
# Reward model scores the full text

def build_text(row):
    prompt = (
        "Describe the properties in the image from the coordinate data.\n"
        "Coordinate data:\n"
        f"{row['prompt']}\n"
        "Description:"
    )
    return prompt + row["completion"]

df["text"] = df.apply(build_text, axis=1)

# ============================================================
# 3. Train / validation split
# ============================================================
dataset = Dataset.from_pandas(df[["text", "reward"]])
dataset = dataset.train_test_split(test_size=0.1, seed=42)

# ============================================================
# 4. Tokenizer
# ============================================================
model_name = "Qwen/Qwen3-0.6B"

tokenizer = AutoTokenizer.from_pretrained(model_name)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

def tokenize_fn(batch):
    return tokenizer(
        batch["text"],
        truncation=True,
        max_length=2048,
    )

tokenized = dataset.map(tokenize_fn, batched=True)

# ============================================================
# 5. Reward model
# ============================================================
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=1,
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
model.config.pad_token_id = tokenizer.pad_token_id

# ============================================================
# 6. Training
# ============================================================
training_args = TrainingArguments(
    output_dir="./reward_model_qwen3_description_completion_only",
    # output_dir="./reward_model_qwen3_description_only",
    learning_rate=2e-5,
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=8,
    num_train_epochs=3,
    logging_steps=10,
    eval_strategy="epoch",
    save_strategy="epoch",
    bf16=True,
    report_to="none",
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized["train"],
    eval_dataset=tokenized["test"],
    tokenizer=tokenizer,
    data_collator=DataCollatorWithPadding(tokenizer),
)

trainer.train()

# ============================================================
# 7. Save
# ============================================================
trainer.save_model("./reward_model_qwen3_description_completion_only")
tokenizer.save_pretrained("./reward_model_qwen3_description_completion_only")
# trainer.save_model("./reward_model_qwen3_description_only")
# tokenizer.save_pretrained("./reward_model_qwen3_description_only")
```

```
Loaded rows: 1000
Columns: ['prompt', 'completion', 'reward']
```

```
Map:   0%|          | 0/900 [00:00<?, ? examples/s]
Map: 100%|##########| 900/900 [00:00<00:00, 1000.00 examples/s]
```

```
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.09}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.18}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.27}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.36}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.44}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.53}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.62}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.71}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.8}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.89}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.98}
{'eval_loss': 0.0, 'eval_runtime': 0.2, 'eval_samples_per_second': 500.0, 'eval_steps_per_second': 500.0, 'epoch': 1.0}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.07}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.16}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.25}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.34}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.43}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.51}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.6}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.69}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.78}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.87}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.96}
{'eval_loss': 0.0, 'eval_runtime': 0.2, 'eval_samples_per_second': 500.0, 'eval_steps_per_second': 500.0, 'epoch': 2.0}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.05}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.14}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.23}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.32}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.41}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.49}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.58}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.67}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.76}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.85}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.94}
{'eval_loss': 0.0, 'eval_runtime': 0.2, 'eval_samples_per_second': 500.0, 'eval_steps_per_second': 500.0, 'epoch': 3.0}
{'train_runtime': 3.0, 'train_samples_per_second': 900.0, 'train_steps_per_second': 3.0, 'train_loss': 0.0, 'epoch': 3.0}
```

```
# Cell 3
import os
import torch
import pandas as pd
from datasets import Dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    DataCollatorWithPadding,
)

# ============================================================
# 1. Load CSV
# ============================================================
csv_path = "./reward_data_description_only.csv"
df = pd.read_csv(csv_path)

print("Loaded rows:", len(df))
print("Columns:", df.columns.tolist())

# ============================================================
# 2. Build prompt + completion text
# ============================================================
# Prompt = the same instruction + coordinate data used during RLOO generation
# Completion = the model's description
# Reward model scores the full text

def build_text(row):
    prompt = (
        "Describe the properties in the image from the coordinate data.\n"
        "Coordinate data:\n"
        f"{row['prompt']}\n"
        "Description:"
    )
    return prompt + row["completion"]

df["text"] = df.apply(build_text, axis=1)

# ============================================================
# 3. Train / validation split
# ============================================================
dataset = Dataset.from_pandas(df[["text", "reward"]])
dataset = dataset.train_test_split(test_size=0.1, seed=42)

# ============================================================
# 4. Tokenizer
# ============================================================
model_name = "Qwen/Qwen3-0.6B"

tokenizer = AutoTokenizer.from_pretrained(model_name)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

def tokenize_fn(batch):
    return tokenizer(
        batch["text"],
        truncation=True,
        max_length=2048,
    )

tokenized = dataset.map(tokenize_fn, batched=True)

# ============================================================
# 5. Reward model
# ============================================================
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=1,
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
model.config.pad_token_id = tokenizer.pad_token_id

# ============================================================
# 6. Training
# ============================================================
training_args = TrainingArguments(
    output_dir="./reward_model_qwen3_description_only",
    learning_rate=2e-5,
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=8,
    num_train_epochs=3,
    logging_steps=10,
    eval_strategy="epoch",
    save_strategy="epoch",
    bf16=True,
    report_to="none",
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized["train"],
    eval_dataset=tokenized["test"],
    tokenizer=tokenizer,
    data_collator=DataCollatorWithPadding(tokenizer),
)

trainer.train()

# ============================================================
# 7. Save
# ============================================================
trainer.save_model("./reward_model_qwen3_description_only")
tokenizer.save_pretrained("./reward_model_qwen3_description_only")
```

```
Loaded rows: 1000
Columns: ['prompt', 'completion', 'reward']
```

```
Map:   0%|          | 0/900 [00:00<?, ? examples/s]
Map: 100%|##########| 900/900 [00:00<00:00, 1000.00 examples/s]
```

```
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.09}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.18}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.27}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.36}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.44}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.53}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.62}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.71}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.8}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.89}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.98}
{'eval_loss': 0.0, 'eval_runtime': 0.2, 'eval_samples_per_second': 500.0, 'eval_steps_per_second': 500.0, 'epoch': 1.0}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.07}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.16}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.25}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.34}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.43}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.51}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.6}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.69}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.78}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.87}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.96}
{'eval_loss': 0.0, 'eval_runtime': 0.2, 'eval_samples_per_second': 500.0, 'eval_steps_per_second': 500.0, 'epoch': 2.0}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.05}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.14}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.23}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.32}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.41}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.49}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.58}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.67}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.76}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.85}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.94}
{'eval_loss': 0.0, 'eval_runtime': 0.2, 'eval_samples_per_second': 500.0, 'eval_steps_per_second': 500.0, 'epoch': 3.0}
{'train_runtime': 3.0, 'train_samples_per_second': 900.0, 'train_steps_per_second': 3.0, 'train_loss': 0.0, 'epoch': 3.0}
```

```
# Cell 4
import os
import torch
import pandas as pd
from datasets import Dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    DataCollatorWithPadding,
)

# ============================================================
# 1. Load CSV
# ============================================================
csv_path = "./reward_data_completion_only.csv"
df = pd.read_csv(csv_path)

print("Loaded rows:", len(df))
print("Columns:", df.columns.tolist())

# ============================================================
# 2. Build prompt + completion text
# ============================================================
# Prompt = the same instruction + coordinate data used during RLOO generation
# Completion = the model's description
# Reward model scores the full text

def build_text(row):
    prompt = (
        "Describe the properties in the image from the coordinate data.\n"
        "Coordinate data:\n"
        f"{row['prompt']}\n"
        "Description:"
    )
    return prompt + row["completion"]

df["text"] = df.apply(build_text, axis=1)

# ============================================================
# 3. Train / validation split
# ============================================================
dataset = Dataset.from_pandas(df[["text", "reward"]])
dataset = dataset.train_test_split(test_size=0.1, seed=42)

# ============================================================
# 4. Tokenizer
# ============================================================
model_name = "Qwen/Qwen3-0.6B"

tokenizer = AutoTokenizer.from_pretrained(model_name)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

def tokenize_fn(batch):
    return tokenizer(
        batch["text"],
        truncation=True,
        max_length=2048,
    )

tokenized = dataset.map(tokenize_fn, batched=True)

# ============================================================
# 5. Reward model
# ============================================================
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=1,
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
model.config.pad_token_id = tokenizer.pad_token_id

# ============================================================
# 6. Training
# ============================================================
training_args = TrainingArguments(
    output_dir="./reward_model_qwen3_completion_only",
    learning_rate=2e-5,
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=8,
    num_train_epochs=3,
    logging_steps=10,
    eval_strategy="epoch",
    save_strategy="epoch",
    bf16=True,
    report_to="none",
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized["train"],
    eval_dataset=tokenized["test"],
    tokenizer=tokenizer,
    data_collator=DataCollatorWithPadding(tokenizer),
)

trainer.train()

# ============================================================
# 7. Save
# ============================================================
trainer.save_model("./reward_model_qwen3_completion_only")
tokenizer.save_pretrained("./reward_model_qwen3_completion_only")
```

```
Loaded rows: 1000
Columns: ['prompt', 'completion', 'reward']
```

```
Map:   0%|          | 0/900 [00:00<?, ? examples/s]
Map: 100%|##########| 900/900 [00:00<00:00, 1000.00 examples/s]
```

```
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.09}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.18}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.27}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.36}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.44}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.53}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.62}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.71}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.8}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.89}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.98}
{'eval_loss': 0.0, 'eval_runtime': 0.2, 'eval_samples_per_second': 500.0, 'eval_steps_per_second': 500.0, 'epoch': 1.0}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.07}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.16}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.25}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.34}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.43}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.51}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.6}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.69}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.78}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.87}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.96}
{'eval_loss': 0.0, 'eval_runtime': 0.2, 'eval_samples_per_second': 500.0, 'eval_steps_per_second': 500.0, 'epoch': 2.0}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.05}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.14}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.23}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.32}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.41}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.49}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.58}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.67}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.76}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.85}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.94}
{'eval_loss': 0.0, 'eval_runtime': 0.2, 'eval_samples_per_second': 500.0, 'eval_steps_per_second': 500.0, 'epoch': 3.0}
{'train_runtime': 3.0, 'train_samples_per_second': 900.0, 'train_steps_per_second': 3.0, 'train_loss': 0.0, 'epoch': 3.0}
```

```
# Cell 5
import os
import torch
import pandas as pd
from datasets import Dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    DataCollatorWithPadding,
)

# ============================================================
# 1. Load CSV
# ============================================================
csv_path = "./reward_data_completion_only.csv"
df = pd.read_csv(csv_path)

print("Loaded rows:", len(df))
print("Columns:", df.columns.tolist())

# ============================================================
# 2. Build prompt + completion text
# ============================================================
# Prompt = the same instruction + coordinate data used during RLOO generation
# Completion = the model's description
# Reward model scores the full text

def build_text(row):
    prompt = (
        "Describe the properties in the image from the coordinate data.\n"
        "Coordinate data:\n"
        f"{row['prompt']}\n"
        "Description:"
    )
    return prompt + row["completion"]

df["text"] = df.apply(build_text, axis=1)

# ============================================================
# 3. Train / validation split
# ============================================================
dataset = Dataset.from_pandas(df[["text", "reward"]])
dataset = dataset.train_test_split(test_size=0.1, seed=42)

# ============================================================
# 4. Tokenizer
# ============================================================
model_name = "Qwen/Qwen3-0.6B"

tokenizer = AutoTokenizer.from_pretrained(model_name)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

def tokenize_fn(batch):
    return tokenizer(
        batch["text"],
        truncation=True,
        max_length=2048,
    )

tokenized = dataset.map(tokenize_fn, batched=True)

# ============================================================
# 5. Reward model
# ============================================================
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=1,
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
model.config.pad_token_id = tokenizer.pad_token_id

# ============================================================
# 6. Training
# ============================================================
training_args = TrainingArguments(
    output_dir="./reward_model_qwen3_completion_only",
    learning_rate=2e-5,
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=8,
    num_train_epochs=3,
    logging_steps=10,
    eval_strategy="epoch",
    save_strategy="epoch",
    bf16=True,
    report_to="none",
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized["train"],
    eval_dataset=tokenized["test"],
    tokenizer=tokenizer,
    data_collator=DataCollatorWithPadding(tokenizer),
)

trainer.train()

# ============================================================
# 7. Save
# ============================================================
trainer.save_model("./reward_model_qwen3_completion_only")
tokenizer.save_pretrained("./reward_model_qwen3_completion_only")
```

```
Loaded rows: 1000
Columns: ['prompt', 'completion', 'reward']
```

```
Map:   0%|          | 0/900 [00:00<?, ? examples/s]
Map: 100%|##########| 900/900 [00:00<00:00, 1000.00 examples/s]
```

```
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.09}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.18}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.27}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.36}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.44}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.53}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.62}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.71}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.8}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.89}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 0.98}
{'eval_loss': 0.0, 'eval_runtime': 0.2, 'eval_samples_per_second': 500.0, 'eval_steps_per_second': 500.0, 'epoch': 1.0}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.07}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.16}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.25}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.34}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.43}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.51}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.6}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.69}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.78}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.87}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 1.96}
{'eval_loss': 0.0, 'eval_runtime': 0.2, 'eval_samples_per_second': 500.0, 'eval_steps_per_second': 500.0, 'epoch': 2.0}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.05}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.14}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.23}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.32}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.41}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.49}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.58}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.67}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.76}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.85}
{'loss': 0.0, 'grad_norm': 0.0, 'learning_rate': 2e-05, 'epoch': 2.94}
{'eval_loss': 0.0, 'eval_runtime': 0.2, 'eval_samples_per_second': 500.0, 'eval_steps_per_second': 500.0, 'epoch': 3.0}
{'train_runtime': 3.0, 'train_samples_per_second': 900.0, 'train_steps_per_second': 3.0, 'train_loss': 0.0, 'epoch': 3.0}
```

```
# Cell 6
import os
import torch
import pandas as pd
from datasets import Dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    DataCollatorWithPadding,
)

# ============================================================
# 1. Load CSV
# ============================================================
csv_path = "./reward_data_description_only.csv"
df = pd.read_csv(csv_path)

print("Loaded rows:", len(df))
print("Columns:", df.columns.tolist())

# ============================================================
# 2. Build prompt + completion text
# ============================================================
# Prompt = the same instruction + coordinate data used during RLOO generation
# Completion = the model's description
# Reward model scores the full text

def build_text(row):
    prompt = (
        "Describe the properties in the image from the coordinate data.\n"
        "Coordinate data:\n"
        f"{row['prompt']}\n"
        "Description:"
    )
    return prompt + row["completion"]

df["text"] = df.apply(build_text, axis=1)

# ============================================================
# 3. Train / validation split
# ============================================================
dataset = Dataset.from_pandas(df[["text", "reward"]])
dataset = dataset.train_test_split(test_size=0.1, seed=42)

# ============================================================
# 4. Tokenizer
# ============================================================
model_name = "Qwen/Qwen3-0.6B"

tokenizer = AutoTokenizer.from_pretrained(model_name)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

def tokenize_fn(batch):
    return tokenizer(
        batch["text"],
        truncation=True,
        max_length=2048,
    )

tokenized = dataset.map(tokenize_f
```


		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00

Table 2.

Name	Address	Column 3	Column 4	Column 5	Column 6
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00
		11.00	11.00	11.00	11.00

Model	Loss	Train Accuracy	Test Accuracy	Precision	Recall	F1 Score
Model_1	0.00	0.00	0.00	0.00	0.00	0.00
Model_2	0.00	0.00	0.00	0.00	0.00	0.00
Model_3	0.00	0.00	0.00	0.00	0.00	0.00
Model_4	0.00	0.00	0.00	0.00	0.00	0.00
Model_5	0.00	0.00	0.00	0.00	0.00	0.00
Model_6	0.00	0.00	0.00	0.00	0.00	0.00
Model_7	0.00	0.00	0.00	0.00	0.00	0.00
Model_8	0.00	0.00	0.00	0.00	0.00	0.00
Model_9	0.00	0.00	0.00	0.00	0.00	0.00
Model_10	0.00	0.00	0.00	0.00	0.00	0.00

Model	Loss	Train Accuracy	Test Accuracy	Precision	Recall	F1 Score
Model_1	0.00	0.00	0.00	0.00	0.00	0.00
Model_2	0.00	0.00	0.00	0.00	0.00	0.00
Model_3	0.00	0.00	0.00	0.00	0.00	0.00
Model_4	0.00	0.00	0.00	0.00	0.00	0.00
Model_5	0.00	0.00	0.00	0.00	0.00	0.00
Model_6	0.00	0.00	0.00	0.00	0.00	0.00
Model_7	0.00	0.00	0.00	0.00	0.00	0.00
Model_8	0.00	0.00	0.00	0.00	0.00	0.00
Model_9	0.00	0.00	0.00	0.00	0.00	0.00
Model_10	0.00	0.00	0.00	0.00	0.00	0.00

Model	Loss	Train Accuracy	Test Accuracy	Precision	Recall	F1 Score
Model_1	0.00	0.00	0.00	0.00	0.00	0.00
Model_2	0.00	0.00	0.00	0.00	0.00	0.00
Model_3	0.00	0.00	0.00	0.00	0.00	0.00
Model_4	0.00	0.00	0.00	0.00	0.00	0.00
Model_5	0.00	0.00	0.00	0.00	0.00	0.00
Model_6	0.00	0.00	0.00	0.00	0.00	0.00
Model_7	0.00	0.00	0.00	0.00	0.00	0.00
Model_8	0.00	0.00	0.00	0.00	0.00	0.00
Model_9	0.00	0.00	0.00	0.00	0.00	0.00
Model_10	0.00	0.00	0.00	0.00	0.00	0.00

Model	Response	Cross-Domain	Structural Analogy	Abstract Pattern	Novelty	Overthinking
Qwen 3 8B	<think> Okay, so I need to figure out how to design a new kind of computer that's not like the ones we have now. Let me start by thinking about what makes current computers work. They use binary, right? Zeros and ones, transistors, logic gates. But maybe there's a different way to process information. Hmm, maybe something that's more like how the brain works? Or maybe using different physical properties? Wait, the user wants a computer that's not based on binary. So I need to think of alternatives. Let me brainstorm some ideas. Quantum computers use qubits, but they're still binary in a way. Maybe something with more states? Like ternary computing? But that's still discrete. Or maybe analog computing? But analog is usually for continuous signals, not digital. Another thought: maybe using chemical reactions? Or biological systems? Like DNA computing? But that's more for specific problems. Or maybe using light in a different way, like optical computing? But that's still binary. Wait, the user mentioned "not based on binary". So perhaps something that uses a different kind of logic. Maybe fuzzy logic? But that's still digital. Or maybe something with probabilities? Like probabilistic computing? But that's also digital. Alternatively, maybe using a different physical medium. For example, using the properties of materials at the atomic level. Or maybe using the flow of fluids? Like hydraulic computers? But that's not practical for modern applications. Let me think about the components. Instead of transistors, what else can be used? Maybe memristors? They can remember their state, which is useful for neural networks. But again, that's still binary. Wait, the user wants a computer that's not based on binary. So maybe something that uses a different kind of state. For example, using the spin of electrons in a way that's not just up or down. But that's still binary. Alternatively, maybe using a different kind of logic gate. For example, using the properties of superconductors? Or maybe using the interference of waves? Like in optical computing, but again, that's still binary. Hmm, maybe I need to think of a completely different approach. Let me consider the user's request again. They want a computer that's not based on binary. So perhaps something that uses a different kind of information processing. Maybe using the properties of the universe itself? Like using the laws of physics to perform computations? Wait, there's a concept called "reversible computing" where you don't lose information, but that's still binary. Or maybe "quantum computing" but that's still binary in a way. Maybe "analog computing" but that's not digital. Alternatively, maybe using a different kind of number system. For example, using a base-3 system (ternary) instead of base-2. But that's still digital. Or maybe using a base-10 system? But that's also digital. Wait, the user wants a computer that's not based on binary. So maybe something that uses a different kind of state. For example, using the position of a particle in space? Like using the location of a particle to represent information. But that's still binary. Alternatively, maybe using the frequency of a wave

Ipsum	Dolor sit amet	Consectetur	Adipiscing	Elit	Sed
Eiusmod	0.00	+0.00	+0.00	+0.00	0.00
Tempor	0.00	+0.00	+0.00	+0.00	0.00
Incididunt	0.00	+0.00	+0.00	+0.00	0.00
Labore	0.00	+0.00	+0.00	+0.00	0.00
Dolore	0.00	+0.00	+0.00	+0.00	0.00
Magna	0.00	+0.00	+0.00	+0.00	0.00
Aliqua	0.00	+0.00	+0.00	+0.00	0.00
Ut	0.00	+0.00	+0.00	+0.00	0.00
Enim	0.00	+0.00	+0.00	+0.00	0.00
Ad	0.00	+0.00	+0.00	+0.00	0.00
Minim	0.00	+0.00	+0.00	+0.00	0.00
Veniam	0.00	+0.00	+0.00	+0.00	0.00
Quis	0.00	+0.00	+0.00	+0.00	0.00
Nostrud	0.00	+0.00	+0.00	+0.00	0.00
Exercitation	0.00	+0.00	+0.00	+0.00	0.00
Ullamco	0.00	+0.00	+0.00	+0.00	0.00
Laboris	0.00	+0.00	+0.00	+0.00	0.00
Nisi	0.00	+0.00	+0.00	+0.00	0.00
Aliquip	0.00	+0.00	+0.00	+0.00	0.00
Ex	0.00	+0.00	+0.00	+0.00	0.00
Ea	0.00	+0.00	+0.00	+0.00	0.00
Commodo	0.00	+0.00	+0.00	+0.00	0.00
Consequat	0.00	+0.00	+0.00	+0.00	0.00
Duis	0.00	+0.00	+0.00	+0.00	0.00
Aute	0.00	+0.00	+0.00	+0.00	0.00
Irure	0.00	+0.00	+0.00	+0.00	0.00
In	0.00	+0.00	+0.00	+0.00	0.00
Reprehenderit	0.00	+0.00	+0.00	+0.00	0.00
Voluptate	0.00	+0.00	+0.00	+0.00	0.00
Velit	0.00	+0.00	+0.00	+0.00	0.00
Esse	0.00	+0.00	+0.00	+0.00	0.00
Cillum	0.00	+0.00	+0.00	+0.00	0.00
Eu	0.00	+0.00	+0.00	+0.00	0.00
Fugiat	0.00	+0.00	+0.00	+0.00	0.00
Nulla	0.00	+0.00	+0.00	+0.00	0.00
Pariatur	0.00	+0.00	+0.00	+0.00	0.00
Excepteur	0.00	+0.00	+0.00	+0.00	0.00
Sint	0.00	+0.00	+0.00	+0.00	0.00
Occaecat	0.00	+0.00	+0.00	+0.00	0.00
Cupidatat	0.00	+0.00	+0.00	+0.00	0.00
Non	0.00	+0.00	+0.00	+0.00	0.00
Proident	0.00	+0.00	+0.00	+0.00	0.00
Sunt	0.00	+0.00	+0.00	+0.00	0.00
Culpa	0.00	+0.00	+0.00	+0.00	0.00
Qui	0.00	+0.00	+0.00	+0.00	0.00
Officia	0.00	+0.00	+0.00	+0.00	0.00
Deserunt	0.00	+0.00	+0.00	+0.00	0.00
Mollit	0.00	+0.00	+0.00	+0.00	0.00
Anim	0.00	+0.00	+0.00	+0.00	0.00
Id	0.00	+0.00	+0.00	+0.00	0.00
Est	0.00	+0.00	+0.00	+0.00	0.00
Laborum	0.00	+0.00	+0.00	+0.00	0.00

Array 30

Ipsum	Dolor sit amet	Consectetur	Adipiscing	Elit	Sed
Eiusmod	0.00	+0.00	+0.00	+0.00	0.00
Tempor	0.00	+0.00	+0.00	+0.00	0.00
Incididunt	0.00	+0.00	+0.00	+0.00	0.00
Labore	0.00	+0.00	+0.00	+0.00	0.00
Dolore	0.00	+0.00	+0.00	+0.00	0.00
Magna	0.00	+0.00	+0.00	+0.00	0.00
Aliqua	0.00	+0.00	+0.00	+0.00	0.00
Ut	0.00	+0.00	+0.00	+0.00	0.00
Enim	0.00	+0.00	+0.00	+0.00	0.00
Ad	0.00	+0.00	+0.00	+0.00	0.00
Minim	0.00	+0.00	+0.00	+0.00	0.00
Veniam	0.00	+0.00	+0.00	+0.00	0.00
Quis	0.00	+0.00	+0.00	+0.00	0.00
Nostrud	0.00	+0.00	+0.00	+0.00	0.00
Exercitation	0.00	+0.00	+0.00	+0.00	0.00
Ullamco	0.00	+0.00	+0.00	+0.00	0.00
Laboris	0.00	+0.00	+0.00	+0.00	0.00
Nisi	0.00	+0.00	+0.00	+0.00	0.00
Aliquip	0.00	+0.00	+0.00	+0.00	0.00
Ex	0.00	+0.00	+0.00	+0.00	0.00
Ea	0.00	+0.00	+0.00	+0.00	0.00
Commodo	0.00	+0.00	+0.00	+0.00	0.00
Consequat	0.00	+0.00	+0.00	+0.00	0.00
Duis	0.00	+0.00	+0.00	+0.00	0.00
Aute	0.00	+0.00	+0.00	+0.00	0.00
Irure	0.00	+0.00	+0.00	+0.00	0.00
In	0.00	+0.00	+0.00	+0.00	0.00
Reprehenderit	0.00	+0.00	+0.00	+0.00	0.00
Voluptate	0.00	+0.00	+0.00	+0.00	0.00
Velit	0.00	+0.00	+0.00	+0.00	0.00
Esse	0.00	+0.00	+0.00	+0.00	0.00
Cillum	0.00	+0.00	+0.00	+0.00	0.00
Eu	0.00	+0.00	+0.00	+0.00	0.00
Fugiat	0.00	+0.00	+0.00	+0.00	0.00
Nulla	0.00	+0.00	+0.00	+0.00	0.00
Pariatur	0.00	+0.00	+0.00	+0.00	0.00
Excepteur	0.00	+0.00	+0.00	+0.00	0.00
Sint	0.00	+0.00	+0.00	+0.00	0.00
Occaecat	0.00	+0.00	+0.00	+0.00	0.00
Cupidatat	0.00	+0.00	+0.00	+0.00	0.00
Non	0.00	+0.00	+0.00	+0.00	0.00
Proident	0.00	+0.00	+0.00	+0.00	0.00
Sunt	0.00	+0.00	+0.00	+0.00	0.00
Culpa	0.00	+0.00	+0.00	+0.00	0.00
Qui	0.00	+0.00	+0.00	+0.00	0.00
Officia	0.00	+0.00	+0.00	+0.00	0.00
Deserunt	0.00	+0.00	+0.00	+0.00	0.00
Mollit	0.00	+0.00	+0.00	+0.00	0.00
Anim	0.00	+0.00	+0.00	+0.00	0.00
Id	0.00	+0.00	+0.00	+0.00	0.00
Est	0.00	+0.00	+0.00	+0.00	0.00
Laborum	0.00	+0.00	+0.00	+0.00	0.00

Array 31

Ipsum	Dolor sit amet	Consectetur	Adipiscing	Elit	Sed
Eiusmod	0.00	+0.00	+0.00	+0.00	0.00
Tempor	0.00	+0.00	+0.00	+0.00	0.00
Incididunt	0.00	+0.00	+0.00	+0.00	0.00
Labore	0.00	+0.00	+0.00	+0.00	0.00
Dolore	0.00	+0.00	+0.00	+0.00	0.00
Magna	0.00	+0.00	+0.00	+0.00	0.00
Aliqua	0.00	+0.00	+0.00	+0.00	0.00
Ut	0.00	+0.00	+0.00	+0.00	0.00
Enim	0.00	+0.00	+0.00	+0.00	0.00
Ad	0.00	+0.00	+0.00	+0.00	0.00
Minim	0.00	+0.00	+0.00	+0.00	0.00
Veniam	0.00	+0.00	+0.00	+0.00	0.00
Quis	0.00	+0.00	+0.00	+0.00	0.00
Nostrud	0.00	+0.00	+0.00	+0.00	0.00
Exercitation	0.00	+0.00	+0.00	+0.00	0.00
Ullamco	0.00	+0.00	+0.00	+0.00	0.00
Laboris	0.00	+0.00	+0.00	+0.00	0.00
Nisi	0.00	+0.00	+0.00	+0.00	0.00
Aliquip	0.00	+0.00	+0.00	+0.00	0.00
Ex	0.00	+0.00	+0.00	+0.00	0.00
Ea	0.00	+0.00	+0.00	+0.00	0.00
Commodo	0.00	+0.00	+0.00	+0.00	0.00
Consequat	0.00	+0.00	+0.00	+0.00	0.00
Duis	0.00	+0.00	+0.00	+0.00	0.00
Aute	0.00	+0.00	+0.00	+0.00	0.00
Irure	0.00	+0.00	+0.00	+0.00	0.00
In	0.00	+0.00	+0.00	+0.00	0.00
Reprehenderit	0.00	+0.00	+0.00	+0.00	0.00
Voluptate	0.00	+0.00	+0.00	+0.00	0.00
Velit	0.00	+0.00	+0.00	+0.00	0.00
Esse	0.00	+0.00	+0.00	+0.00	0.00
Cillum	0.00	+0.00	+0.00	+0.00	0.00
Eu	0.00	+0.00	+0.00	+0.00	0.00
Fugiat	0.00	+0.00	+0.00	+0.00	0.00
Nulla	0.00	+0.00	+0.00	+0.00	0.00
Pariatur	0.00	+0.00	+0.00	+0.00	0.00
Excepteur	0.00	+0.00	+0.00	+0.00	0.00
Sint	0.00	+0.00	+0.00	+0.00	0.00
Occaecat	0.00	+0.00	+0.00	+0.00	0.00
Cupidatat	0.00	+0.00	+0.00	+0.00	0.00
Non	0.00	+0.00	+0.00	+0.00	0.00
Proident	0.00	+0.00	+0.00	+0.00	0.00
Sunt	0.00	+0.00	+0.00	+0.00	0.00
Culpa	0.00	+0.00	+0.00	+0.00	0.00
Qui	0.00	+0.00	+0.00	+0.00	0.00
Officia	0.00	+0.00	+0.00	+0.00	0.00
Deserunt	0.00	+0.00	+0.00	+0.00	0.00
Mollit	0.00	+0.00	+0.00	+0.00	0.00
Anim	0.00	+0.00	+0.00	+0.00	0.00
Id	0.00	+0.00	+0.00	+0.00	0.00
Est	0.00	+0.00	+0.00	+0.00	0.00
Laborum	0.00	+0.00	+0.00	+0.00	0.00

Array 32

Ipsum	Dolor sit amet	Consectetur	Adipiscing	Elit	Sed
Eiusmod	0.00	+0.00	+0.00	+0.00	0.00
Tempor	0.00	+0.00	+0.00	+0.00	0.00
Incididunt	0.00	+0.00	+0.00	+0.00	0.00
Labore	0.00	+0.00	+0.00	+0.00	0.00
Dolore	0.00	+0.00	+0.00	+0.00	0.00
Magna	0.00	+0.00	+0.00	+0.00	0.00
Aliqua	0.00	+0.00	+0.00	+0.00	0.00
Ut	0.00	+0.00	+0.00	+0.00	0.00
Enim	0.00	+0.00	+0.00	+0.00	0.00
Ad	0.00	+0.00	+0.00	+0.00	0.00
Minim	0.00	+0.00	+0.00	+0.00	0.00
Veniam	0.00	+0.00	+0.00	+0.00	0.00
Quis	0.00	+0.00	+0.00	+0.00	0.00
Nostrud	0.00	+0.00	+0.00	+0.00	0.00
Exercitation	0.00	+0.00	+0.00	+0.00	0.00
Ullamco	0.00	+0.00	+0.00	+0.00	0.00
Laboris	0.00	+0.00	+0.00	+0.00	0.00
Nisi	0.00	+0.00	+0.00	+0.00	0.00
Aliquip	0.00	+0.00	+0.00	+0.00	0.00
Ex	0.00	+0.00	+0.00	+0.00	0.00
Ea	0.00	+0.00	+0.00	+0.00	0.00
Commodo	0.00	+0.00	+0.00	+0.00	0.00
Consequat	0.00	+0.00	+0.00	+0.00	0.00
Duis	0.00	+0.00	+0.00	+0.00	0.00
Aute	0.00	+0.00	+0.00	+0.00	0.00
Irure	0.00	+0.00	+0.00	+0.00	0.00
In	0.00	+0.00	+0.00	+0.00	0.00
Reprehenderit	0.00	+0.00	+0.00	+0.00	0.00
Voluptate	0.00	+0.00	+0.00	+0.00	0.00
Velit	0.00	+0.00	+0.00	+0.00	0.00
Esse	0.00	+0.00	+0.00	+0.00	0.00
Cillum	0.00	+0.00	+0.00	+0.00	0.00
Eu	0.00	+0.00	+0.00	+0.00	0.00
Fugiat	0.00	+0.00	+0.00	+0.00	0.00
Nulla	0.00	+0.00	+0.00	+0.00	0.00
Pariatur	0.00	+0.00	+0.00	+0.00	0.00
Excepteur	0.00	+0.00	+0.00	+0.00	0.00
Sint	0.00	+0.00	+0.00	+0.00	0.00
Occaecat	0.00	+0.00	+0.00	+0.00	0.00
Cupidatat	0.00	+0.00	+0.00	+0.00	0.00
Non	0.00	+0.00	+0.00	+0.00	0.00
Proident	0.00	+0.00	+0.00	+0.00	0.00
Sunt	0.00	+0.00	+0.00	+0.00	0.00
Culpa	0.00	+0.00	+0.00	+0.00	0.00
Qui	0.00	+0.00	+0.00	+0.00	0.00
Officia	0.00	+0.00	+0.00	+0.00	0.00
Deserunt	0.00	+0.00	+0.00	+0.00	0.00
Mollit	0.00	+0.00	+0.00	+0.00	0.00
Anim	0.00	+0.00	+0.00	+0.00	0.00
Id	0.00	+0.00	+0.00	+0.00	0.00
Est	0.00	+0.00	+0.00	+0.00	0.00
Laborum	0.00	+0.00	+0.00	+0.00	0.00

Array 33

Ipsum	Dolor sit amet	Consectetur	Adipiscing	Elit	Sed
Eiusmod	0.00	+0.00	+0.00	+0.00	0.00
Tempor	0.00	+0.00	+0.00	+0.00	0.00
Incididunt	0.00	+0.00	+0.00	+0.00	0.00
Labore	0.00	+0.00	+0.00	+0.00	0.00
Dolore	0.00	+0.00	+0.00	+0.00	0.00
Magna	0.00	+0.00	+0.00	+0.00	0.00
Aliqua	0.00	+0.00	+0.00	+0.00	0.00
Ut	0.00	+0.00	+0.00	+0.00	0.00
Enim	0.00	+0.00	+0.00	+0.00	0.00
Ad	0.00	+0.00	+0.00	+0.00	0.00
Minim	0.00	+0.00	+0.00	+0.00	0.00
Veniam	0.00	+0.00	+0.00	+0.00	0.00
Quis	0.00	+0.00	+0.00	+0.00	0.00
Nostrud	0.00	+0.00	+0.00	+0.00	0.00
Exercitation	0.00	+0.00	+0.00	+0.00	0.00
Ullamco	0.00	+0.00	+0.00	+0.00	0.00
Laboris	0.00	+0.00	+0.00	+0.00	0.00
Nisi	0.00	+0.00	+0.00	+0.00	0.00
Aliquip	0.00	+0.00	+0.00	+0.00	0.00
Ex	0.00	+0.00	+0.00	+0.00	0.00
Ea	0.00	+0.00	+0.00	+0.00	0.00
Commodo	0.00	+0.00	+0.00	+0.00	0.00
Consequat	0.00	+0.00	+0.00	+0.00	0.00
Duis	0.00	+0.00	+0.00	+0.00	0.00
Aute	0.00	+0.00	+0.00	+0.00	0.00
Irure	0.00	+0.00	+0.00	+0.00	0.00
In	0.00	+0.00	+0.00	+0.00	0.00
Reprehenderit	0.00	+0.00	+0.00	+0.00	0.00
Voluptate	0.00	+0.00	+0.00	+0.00	0.00
Velit	0.00	+0.00	+0.00	+0.00	0.00
Esse	0.00	+0.00	+0.00	+0.00	0.00
Cillum	0.00	+0.00	+0.00	+0.00	0.00
Eu	0.00	+0.00	+0.00	+0.00	0.00
Fugiat	0.00	+0.00	+0.00	+0.00	0.00
Nulla	0.00	+0.00	+0.00	+0.00	0.00
Pariatur	0.00	+0.00	+0.00	+0.00	0.00
Excepteur	0.00	+0.00	+0.00	+0.00	0.00
Sint	0.00	+0.00	+0.00	+0.00	0.00
Occaecat	0.00	+0.00	+0.00	+0.00	0.00
Cupidatat	0.00	+0.00	+0.00	+0.00	0.00
Non	0.00	+0.00	+0.00	+0.00	0.00
Proident	0.00	+0.00	+0.00	+0.00	0.00
Sunt	0.00	+0.00	+0.00	+0.00	0.00
Culpa	0.00	+0.00	+0.00	+0.00	0.00
Qui	0.00	+0.00	+0.00	+0.00	0.00
Officia	0.00	+0.00	+0.00	+0.00	0.00
Deserunt	0.00	+0.00	+0.00	+0.00	0.00
Mollit	0.00	+0.00	+0.00	+0.00	0.00
Anim	0.00	+0.00	+0.00	+0.00	0.00
Id	0.00	+0.00	+0.00	+0.00	0.00
Est	0.00	+0.00	+0.00	+0.00	0.00
Laborum	0.00	+0.00	+0.00	+0.00	0.00

Array 34

Ipsum	Dolor sit amet	Consectetur	Adipiscing	Elit	Sed
Eiusmod	0.00	+0.00	+0.00	+0.00	0.00
Tempor	0.00	+0.00	+0.00	+0.00	0.00
Incididunt	0.00	+0.00	+0.00	+0.00	0.00
Labore	0.00	+0.00	+0.00	+0.00	0.00
Dolore	0.00	+0.00	+0.00	+0.00	0.00
Magna	0.00	+0.00	+0.00	+0.00	0.00
Aliqua	0.00	+0.00	+0.00	+0.00	0.00
Ut	0.00	+0.00	+0.00	+0.00	0.00
Enim	0.00	+0.00	+0.00	+0.00	0.00
Ad	0.00	+0.00	+0.00	+0.00	0.00
Minim	0.00	+0.00	+0.00	+0.00	0.00
Veniam	0.00	+0.00	+0.00	+0.00	0.00
Quis	0.00	+0.00	+0.00	+0.00	0.00
Nostrud	0.00	+0.00	+0.00	+0.00	0.00
Exercitation	0.00	+0.00	+0.00	+0.00	0.00
Ullamco	0.00	+0.00	+0.00	+0.00	0.00
Laboris	0.00	+0.00	+0.00	+0.00	0.00
Nisi	0.00	+0.00	+0.00	+0.00	0.00
Aliquip	0.00	+0.00	+0.00	+0.00	0.00
Ex	0.00	+0.00	+0.00	+0.00	0.00
Ea	0.00	+0.00	+0.00	+0.00	0.00
Commodo	0.00	+0.00	+0.00	+0.00	0.00
Consequat	0.00	+0.00	+0.00	+0.00	0.00
Duis	0.00	+0.00	+0.00	+0.00	0.00
Aute	0.00	+0.00	+0.00	+0.00	0.00
Irure	0.00	+0.00	+0.00	+0.00	0.00
In	0.00	+0.00	+0.00	+0.00	0.00
Reprehenderit	0.00	+0.00	+0.00	+0.00	0.00
Voluptate	0.00	+0.00	+0.00	+0.00	0.00
Velit	0.00	+0.00	+0.00	+0.00	0.00
Esse	0.00	+0.00	+0.00	+0.00	0.00
Cillum	0.00	+0.00	+0.00	+0.00	0.00
Eu	0.00	+0.00	+0.00	+0.00	0.00
Fugiat	0.00	+0.00	+0.00	+0.00	0.00
Nulla	0.00	+0.00	+0.00	+0.00	0.00
Pariatur	0.00	+0.00	+0.00	+0.00	0.00
Excepteur	0.00	+0.00	+0.00	+0.00	0.00
Sint	0.00	+0.00	+0.00	+0.00	0.00
Occaecat	0.00	+0.00	+0.00	+0.00	0.00
Cupidatat	0.00	+0.00	+0.00	+0.00	0.00
Non	0.00	+0.00	+0.00	+0.00	0.00
Proident	0.00	+0.00	+0.00	+0.00	0.00
Sunt	0.00	+0.00	+0.00	+0.00	0.00
Culpa	0.00	+0.00	+0.00	+0.00	0.00
Qui	0.00	+0.00	+0.00	+0.00	0.00
Officia	0.00	+0.00	+0.00	+0.00	0.00
Deserunt	0.00	+0.00	+0.00	+0.00	0.00
Mollit	0.00	+0.00	+0.00	+0.00	0.00
Anim	0.00	+0.00	+0.00	+0.00	0.00
Id	0.00	+0.00	+0.00	+0.00	0.00
Est	0.00	+0.00	+0.00	+0.00	0.00
Laborum	0.00	+0.00	+0.00	+0.00	0.00

Array 35

Ipsum	Dolor sit amet	Consectetur	Adipiscing	Elit	Sed
Eiusmod	0.00	+0.00	+0.00	+0.00	0.00
Tempor	0.00	+0.00	+0.00	+0.00	0.00
Incididunt	0.00	+0.00	+0.00	+0.00	0.00
Labore	0.00	+0.00	+0.00	+0.00	0.00
Dolore	0.00	+0.00	+0.00	+0.00	0.00
Magna	0.00	+0.00	+0.00	+0.00	0.00
Aliqua	0.00	+0.00	+0.00	+0.00	0.00
Ut	0.00	+0.00	+0.00	+0.00	0.00
Enim	0.00	+0.00	+0.00	+0.00	0.00
Ad	0.00	+0.00	+0.00	+0.00	0.00
Minim	0.00	+0.00	+0.00	+0.00	0.00
Veniam	0.00	+0.00	+0.00	+0.00	0.00
Quis	0.00	+0.00	+0.00	+0.00	0.00
Nostrud	0.00	+0.00	+0.00	+0.00	0.00
Exercitation	0.00	+0.00	+0.00	+0.00	0.00
Ullamco	0.00	+0.00	+0.00	+0.00	0.00
Laboris	0.00	+0.00	+0.00	+0.00	0.00
Nisi	0.00	+0.00	+0.00	+0.00	0.00
Aliquip	0.00	+0.00	+0.00	+0.00	0.00
Ex	0.00	+0.00	+0.00	+0.00	0.00
Ea	0.00	+0.00	+0.00	+0.00	0.00
Commodo	0.00	+0.00	+0.00	+0.00	0.00
Consequat	0.00	+0.00	+0.00	+0.00	0.00
Duis	0.00	+0.00	+0.00	+0.00	0.00
Aute	0.00	+0.00	+0.00	+0.00	0.00
Irure	0.00	+0.00	+0.00	+0.00	0.00
In	0.00	+0.00	+0.00	+0.00	0.00
Reprehenderit	0.00	+0.00	+0.00	+0.00	0.00
Voluptate	0.00	+0.00	+0.00	+0.00	0.00
Velit	0.00	+0.00	+0.00	+0.00	0.00
Esse	0.00	+0.00	+0.00	+0.00	0.00
Cillum	0.00	+0.00	+0.00	+0.00	0.00
Eu	0.00	+0.00	+0.00	+0.00	0.00
Fugiat	0.00	+0.00	+0.00	+0.00	0.00
Nulla	0.00	+0.00	+0.00	+0.00	0.00
Pariatur	0.00	+0.00	+0.00	+0.00	0.00
Excepteur	0.00	+0.00	+0.00	+0.00	0.00
Sint	0.00	+0.00	+0.00	+0.00	0.00
Occaecat	0.00	+0.00	+0.00	+0.00	0.00
Cupidatat	0.00	+0.00	+0.00	+0.00	0.00
Non	0.00	+0.00	+0.00	+0.00	0.00
Proident	0.00	+0.00	+0.00	+0.00	0.00
Sunt	0.00	+0.00	+0.00	+0.00	0.00
Culpa	0.00	+0.00	+0.00	+0.00	0.00
Qui	0.00	+0.00	+0.00	+0.00	0.00
Officia	0.00	+0.00	+0.00	+0.00	0.00
Deserunt	0.00	+0.00	+0.00	+0.00	0.00
Mollit	0.00	+0.00	+0.00	+0.00	0.00
Anim	0.00	+0.00	+0.00	+0.00	0.00
Id	0.00	+0.00	+0.00	+0.00	0.00
Est	0.00	+0.00	+0.00	+0.00	0.00
Laborum	0.00	+0.00	+0.00	+0.00	0.00

Array 36

Ipsum	Dolor sit amet	Consectetur	Adipiscing	Elit	Sed
Eiusmod	0.00	+0.00	+0.00	+0.00	0.00
Tempor	0.00	+0.00	+0.00	+0.00	0.00
Incididunt	0.00	+0.00	+0.00	+0.00	0.00
Labore	0.00	+0.00	+0.00	+0.00	0.00
Dolore	0.00	+0.00	+0.00	+0.00	0.00
Magna	0.00	+0.00	+0.00	+0.00	0.00
Aliqua	0.00	+0.00	+0.00	+0.00	0.00
Ut	0.00	+0.00	+0.00	+0.00	0.00
Enim	0.00	+0.00	+0.00	+0.00	0.00
Ad	0.00	+0.00	+0.00	+0.00	0.00
Minim	0.00	+0.00	+0.00	+0.00	0.00
Veniam	0.00	+0.00	+0.00	+0.00	0.00
Quis	0.00	+0.00	+0.00	+0.00	0.00
Nostrud	0.00	+0.00	+0.00	+0.00	0.00
Exercitation	0.00	+0.00	+0.00	+0.00	0.00
Ullamco	0.00	+0.00	+0.00	+0.00	0.00
Laboris	0.00	+0.00	+0.00	+0.00	0.00
Nisi	0.00	+0.00	+0.00	+0.00	0.00
Aliquip	0.00	+0.00	+0.00	+0.00	0.00
Ex	0.00	+0.00	+0.00	+0.00	0.00
Ea	0.00	+0.00	+0.00	+0.00	0.00
Commodo	0.00	+0.00	+0.00	+0.00	0.00
Consequat	0.00	+0.00	+0.00	+0.00	0.00
Duis	0.00	+0.00	+0.00	+0.00	0.00
Aute	0.00	+0.00	+0.00	+0.00	0.00
Irure	0.00	+0.00	+0.00	+0.00	0.00
In	0.00	+0.00	+0.00	+0.00	0.00
Reprehenderit	0.00	+0.00	+0.00	+0.00	0.00
Voluptate	0.00	+0.00	+0.00	+0.00	0.00
Velit	0.00	+0.00	+0.00	+0.00	0.00
Esse	0.00	+0.00	+0.00	+0.00	0.00
Cillum	0.00	+0.00	+0.00	+0.00	0.00
Eu	0.00	+0.00	+0.00	+0.00	0.00
Fugiat	0.00	+0.00	+0.00	+0.00	0.00
Nulla	0.00	+0.00	+0.00	+0.00	0.00
Pariatur	0.00	+0.00	+0.00	+0.00	0.00
Excepteur	0.00	+0.00	+0.00	+0.00	0.00
Sint	0.00	+0.00	+0.00	+0.00	0.00
Occaecat	0.00	+0.00	+0.00	+0.00	0.00
Cupidatat	0.00	+0.00	+0.00	+0.00	0.00
Non	0.00	+0.00	+0.00	+0.00	0.00
Proident	0.00	+0.00	+0.00	+0.00	0.00
Sunt	0.00	+0.00	+0.00	+0.00	0.00
Culpa	0.00	+0.00	+0.00	+0.00	0.00
Qui	0.00	+0.00	+0.00	+0.00	0.00
Officia	0.00	+0.00	+0.00	+0.00	0.00
Deserunt	0.00	+0.00	+0.00	+0.00	0.00
Mollit	0.00	+0.00	+0.00	+0.00	0.00
Anim	0.00	+0.00	+0.00	+0.00	0.00
Id	0.00	+0.00	+0.00	+0.00	0.00
Est	0.00	+0.00	+0.00	+0.00	0.00
Laborum	0.00	+0.00	+0.00	+0.00	0.00

Array 37

Ipsum	Dolor sit amet	Consectetur	Adipiscing	Elit	Sed
Eiusmod	0.00	+0.00	+0.00	+0.00	0.00
Tempor	0.00	+0.00	+0.00	+0.00	0.00
Incididunt	0.00	+0.00	+0.00	+0.00	0.00
Labore	0.00	+0.00	+0.00	+0.00	0.00
Dolore	0.00

1	1.00	-1.00	-0.50	-0.50	0.50
2	1.00	-1.00	-0.50	-0.50	0.50
3	1.00	-1.00	-0.50	-0.50	0.50
4	1.00	-1.00	-0.50	-0.50	0.50
5	1.00	-1.00	-0.50	-0.50	0.50
6	1.00	-1.00	-0.50	-0.50	0.50
7	1.00	-1.00	-0.50	-0.50	0.50
8	1.00	-1.00	-0.50	-0.50	0.50
9	1.00	-1.00	-0.50	-0.50	0.50
10	1.00	-1.00	-0.50	-0.50	0.50

```
# Create a simple DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
    'B': [10, 20, 30, 40, 50],
    'C': [100, 200, 300, 400, 500]
})

# Display the DataFrame
print("Original DataFrame:")
print(df)

# Add a new column 'D' which is the sum of columns 'A' and 'B'
df['D'] = df['A'] + df['B']

# Display the DataFrame after adding the new column
print("\nDataFrame after adding column 'D':")
print(df)
```

```
# Create a simple DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
    'B': [10, 20, 30, 40, 50],
    'C': [100, 200, 300, 400, 500]
})

# Display the DataFrame
print("Original DataFrame:")
print(df)

# Add a new column 'D' which is the sum of columns 'A' and 'B'
df['D'] = df['A'] + df['B']

# Display the DataFrame after adding the new column
print("\nDataFrame after adding column 'D':")
print(df)
```

- injects external knowledge,
- strong for grounding and provenance,
- does not fundamentally fix weak visual representations.

Method — SFT

- teaches direct target behavior,
- stronger and more persistent than prompting,
- requires labeled demonstrations.

Method — LoRA / QLoRA

- captures much of the benefit of tuning at lower memory cost,
- especially attractive on a single strong GPU,
- requires choices about target modules and rank.

Method — DPO

- optimizes relative preferences rather than only demonstrations,
- bridges supervised tuning and RLHF,
- useful when better-vs-worse judgments are easier to obtain than perfect targets.

Method — RLHF / PPO /RLOO

- most expressive and most complex,
- closest to online reward optimization,
- highest engineering and evaluation burden.

Fine-tuning case study summary table

Case study: `Qwen2.5-VL-3B-Instruct` on **Rare Species**, using reduced image resolution through Qwen's `min_pixels` / `max_pixels` controls.

These are **rough engineering estimates**, not official benchmarks. They assume:

- one single device,
- moderate image-token limits,
- a 3B VLM,
- and a practical lecture/lab style workflow rather than highly optimized distributed training.

Method	Extra labeled data needed	A100 80GB	RTX 5090 32GB	DGX Spark	Practical recommendation
Zero-shot	0	Minutes	Minutes	Minutes	Always do first
Few-shot	3–16 exemplars in prompt	Minutes	Minutes	Minutes	Best cheap upgrade over zero-shot
RAG	0 labeled; ~400–1200 species cards/passages	<1 hour setup/indexing	<1 hour setup/indexing	<1 hour setup/indexing	Best when you want grounded explanations
Full SFT	~5k–20k examples	~8–24 GPU-hours	~12–36 GPU-hours	~1–3+ days	Best raw adaptation, but highest supervised cost
LoRA	~1k–10k examples	~2–8 GPU-hours	~3–12 GPU-hours	~8–24 hours	Best first true fine-tuning method
QLoRA	~1k–10k examples	~3–10 GPU-hours	~4–14 GPU-hours	~10–30 hours	Best memory-efficient fine-tuning path
DPO	~5k–20k chosen/rejected pairs	~4–12 GPU-hours	~6–18 GPU-hours	~12–36 hours	Best post-SFT alignment step
RLOO / RL-based	~1k–5k prompts, multiple rollouts each	~10–50+ GPU-hours	~15–80+ GPU-hours	Small demo scale only	Use as the capstone, not the first training method

References

Below is a compact list of the most important references behind the notebook. All links are clickable.

Core methods

1. **LoRA**
   Hu, E. J., Shen, Y., Wallis, P., et al.
   LoRA: Low-Rank Adaptation of Large Language Models

2. **QLoRA**
   Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.
   QLoRA: Efficient Finetuning of Quantized LLMs

3. **RAG**
   Lewis, P., Perez, E., Piktus, A., et al.
   Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

4. **Dense retrieval**
   Karpukhin, V., Oguz, B., Min, S., et al.
   Dense Passage Retrieval for Open-Domain Question Answering

5. **DPO**
   Rafailov, R., Sharma, A., Mitchell, E., et al.
   Direct Preference Optimization: Your Language Model is Secretly a Reward Model

6. **RLHF / InstructGPT**
   Ouyang, L., Wu, J., Jiang, X., et al.
   Training Language Models to Follow Instructions with Human Feedback

7. **RLOO**
   Ahmadian, A., Cremer, C., Gallé, M., et al.
   Back to Basics: Revisiting REINFORCE-Style Optimization for Learning from Human Feedback in LLMs

8. **PPO**
   Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.
   Proximal Policy Optimization Algorithms

9. **Policy gradient foundations**
   Sutton, R. S., McAllester, D., Singh, S., Mansour, Y.
   Policy Gradient Methods for Reinforcement Learning with Function Approximation

10. **Few-shot prompting / in-context learning**
    Brown, T. B., Mann, B., Ryder, N., et al.
    Language Models are Few-Shot Learners

11. **In-context learning analysis**
    Min, S., Lyu, X., Holtzman, A., et al.
    Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?

12. **Implicit Bayesian view of in-context learning**
    Xie, S. M., Raghunathan, A., Liang, P., Ma, T.
    An Explanation of In-context Learning as Implicit Bayesian Inference

Dataset and biodiversity context

13. **Rare Species dataset card**
    imageomics/rare-species

14. **BioCLIP**
    Stevens, S., Wu, J., Thompson, M. J., et al.
    BioCLIP: A Vision Foundation Model for the Tree of Life

Practical tooling

15. **Qwen2.5-VL model card**
    Qwen/Qwen2.5-VL-3B-Instruct

16. **Transformers tokenizer documentation**
    apply_chat_template and assistant-token masks

17. **Transformers multimodal chat templates**
    Hugging Face Transformers documentation

18. **PEFT LoRA documentation**
    Hugging Face PEFT documentation

19. **TRL SFTTrainer documentation**
    Hugging Face TRL SFTTrainer documentation

20. **TRL DPOTrainer documentation**
    Hugging Face TRL DPOTrainer documentation

21. **TRL RewardTrainer documentation**
    Hugging Face TRL Reward Modeling documentation

22. **TRL RLOOTrainer documentation**
    Hugging Face TRL RLOOTrainer documentation

23. **TRL VLM alignment blog**
    Vision Language Model Alignment in TRL

24. **Qwen assistant-mask discussion**
    Chat template of AutoTokenizer does not work on assistant mask tokens

Recent 2024–2026 papers from major venues

25. **ICLR 2025 — theory for token-level RAG**
    Xu, S., Pang, L., Shen, H., Cheng, X.
    A Theory for Token-Level Harmonization in Retrieval-Augmented Generation

26. **ICLR 2025 — hybrid / faithful RAG**
    Ma, S., Xu, C., Jiang, X., et al.
    Think-on-Graph 2.0: Deep and Faithful Large Language Model Reasoning with Knowledge-guided Retrieval-Augmented Generation

27. **ICLR 2025 — multimodal alignment study**
    Amirloo, E., Fauconnier, J.-P., Roesmann, C., et al.
    Understanding Alignment in Multimodal LLMs: A Comprehensive Study

28. **NeurIPS 2024 — self-rewarding VLM alignment**
    Zhou, Y., Wu, T., Zhang, M., et al.
    Calibrated Self-Rewarding Vision Language Models

29. **NeurIPS 2024 — multi-objective multimodal alignment**
    Yang, K., Gao, Y., Wang, H., et al.
    MetaAligner: Towards Generalizable Multi-Objective Alignment of Vision-Language Models

30. **NeurIPS 2024 — self-training for VLMs**
    Deng, Y., Zhang, X., Xu, Y., et al.
    Enhancing Large Vision Language Models with Self-Training on Image Comprehension

31. **AAAI 2026 — multimodal hallucination alleviation**
    Xu, Y., Chen, Y., He, T., et al.
    Adaptive Hallucination Alleviation in Multimodal Large Language Models: From Strategic Data Selection to Severity-Guided Training

32. **AAAI 2026 — hallucination mitigation via early revision**
    Li, Z., Kong, C., Yu, Y.
    SAVER: Mitigating Hallucinations in Large Vision-Language Models via Style-Aware Visual Early Revision

33. **AAAI 2026 — human preference learning in vision generation**
    Xu, J., Huang, Y., Cheng, J., et al.
    VisionReward: Fine-Grained Multi-Dimensional Human Preference Learning for Image and Video Generation

34. **IJCAI 2025 — survey on visual underuse in MLLMs**
    Jain, A., Vatsa, M., Singh, R.
    Words Over Pixels? Rethinking Vision in Multimodal Large Language Models

35. **IJCAI 2025 — prompt learning for VLM adaptation**
    Lv, S.-L., Chen, Y.-Y., Zhou, Z., Yang, M., Guo, L.-Z.
    BMIP: Bi-directional Modality Interaction Prompt Learning for VLM

36. **IJCAI 2025 — preference optimization for trustworthy MLLMs**
    Jiang, S., Zhang, Y., Chen, R., et al.
    Modality-Fair Preference Optimization for Trustworthy MLLM Alignment

Note on IJCAI 2026

As of the date of this notebook, the IJCAI-ECAI 2026 conference website is public, but the 2026 proceedings are not yet public. That is why the newest IJCAI papers cited here are from 2025.

- IJCAI-ECAI 2026 conference site

Use of Generative AI

Portions of this material were developed with the assistance of **generative artificial intelligence tools.
The author reviewed, edited, and validated all content, including explanations, code, and examples, and assumes full responsibility for accuracy and interpretation.